debiasR vs debiaspy

Updated result: 149 passes; two documented API-export differences.

All numerical article outputs pass their existing tolerances, including the full-LAD frequentist models. The two differences concern the intentional Python public-export policy. This report uses the original Census 2021 OD benchmark and records the remaining Bayesian limitation.

This project is currently hosted at
https://github.com/de-bias/debiaspy/tree/contribution/non-bayesian-parity
Regenerated against the integration branch on 13 October 2026.

Contents

1 Executive result **1**

2 Vignette coverage matrix **1**

3 Provenance and comparison rules **1**

4 Getting Set Up **2**
4.1 Complete output inventory . . . 2
4.2 Selected side-by-side table values . . . 2

5 Measuring Coverage Bias **4**
5.1 Complete output inventory . . . 4
5.2 Rendered figure pairs . . . 4
5.3 Figure-data contracts . . . 6
5.4 Selected side-by-side table values . . . 6

6 Identifying And Explaining Bias **8**
6.1 Complete output inventory . . . 8
6.2 Rendered figure pairs . . . 9
6.3 Figure-data contracts . . . 10
6.4 Selected side-by-side table values . . . 11

7 Adjusting Biases **13**
7.1 Complete output inventory . . . 14
7.2 Selected side-by-side table values . . . 14

8 Validation **17**
8.1 Complete output inventory . . . 17
8.2 Rendered figure pairs . . . 17
8.3 Figure-data contracts . . . 18
8.4 Selected side-by-side table values . . . 18

9 Advanced Bayesian Adjustment **22**
9.1 Complete output inventory . . . 22
9.2 Selected side-by-side table values . . . 23

10 Portable OD and benchmark workflow **26**

11 Claim boundary and exceptions **27**

A Complete 151-check inventory **27**

B Bayesian reference quality and fresh empirical check **31**

C Reduced-form investigation: priors and prediction **32**

D Non-Bayesian issue completion and cleanup **33**
D.1 Issue 12: shared distribution calculations . . . 33
D.2 Issue 13: spatial residual diagnostics . . . 33
D.3 Issue 14: multi-method validation . . . 33
D.4 Issue 15: validation and LISA plots . . . 34
D.5 Legacy and metadata cleanup . . . 34

E Simplification and clean installation **35**
E.1 Removed legacy paths and repeated code . . . 35
E.2 Installation, examples and documentation . . . 35
E.3 Reproduce the report and its checks . . . 35

F Fresh article reproduction **36**

G Method-by-method evidence **37**

H Issue contribution checks **39**

1 Executive result

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

The published article audit reruns the declared R and Python calls with the same pinned inputs. It records **72 paired CSV outputs** and **79 JSON/schema checks**; 149 pass and two export-list checks differ. All scientific table values pass. The side-by-side tables are read from newly regenerated outputs. The Python list has the renamed data loader, omits two R compatibility aliases, and exposes its version. These policy differences do not alter calculations.

2 Vignette coverage matrix

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Page	Workflow	CSV	JSON	Rendered pairs	Figure contracts	Result
v03	Getting Set Up	7	8	0	0	**PASS**
v04	Measuring Coverage Bias	11	12	4	4	**PASS**
v05	Identifying And Explaining Bias	9	10	3	3	**PASS**
v06	Adjusting Biases	17	18	0	0	**PASS**
v07	Validation	14	15	3	7	**PASS**
v09	Advanced Bayesian Adjustment	12	13	0	0	**PASS**

Table 1: Complete published-vignette coverage. Rendered pairs are newly generated exhibits; figure contracts are the committed audit records.

3 Provenance and comparison rules

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Reference	Checked revision
debiasR	`6fa2709ae42de6dc525a4a46182fdfb448b14a3f`
Original R data companion	`e652a1186c705d1edc4ff5427e7e2d5230ef7fe7`
Python data companion	`19d1c5f67f5dcf70fcd765e42fa03d667df496df`
Recorded table code	`6d1a2c9`; fresh checks: `ff41d43`
Bayesian investigation	`48ac156`; separate diagnostic evidence
Runtime	Python 3.11.14; R 4.5.1; prepared scientific dependency stack
Article evidence	`docs/reports/article_comparison.json`; 149 PASS, two API differences

The original Census 2021 ODWP01EW workplace-flow benchmark is retained. Python article outputs were regenerated against retained outputs from the clean pinned R checkout. Focused spatial regressions also execute R calls afresh. Artifact paths below identify outputs in the regeneration workspace. Published check records reside in the JSON files under `docs/reports`.

The filename labels identify the implementation and executed audit or regression test files for the stated scope. Precomputed Bayesian tables cite their ingestion code; they do not imply that the article audit ran a sampler. Figure tests compare data contracts, not rendered pixels.

CSV structures, strings, missingness and schemas are compared exactly. Numeric acceptance uses raw precision and existing per-table tolerances. Display rounding does not determine pass/fail. The renamed Python loader and module paths are adapted in the audit caller; no numerical acceptance thresholds or public model calculations were changed.

The Python companion route also matches the original R companion route for MPD, benchmark, coverage, covariates, distances and support-audit tables in five-area LAD and MSOA probes. Tests verify that the Python route invokes no R. Independent validation of external source accuracy is beyond this comparison.

4 Getting Set Up

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v03-getting-set-up.html

Checks the article input objects, row counts, portable schemas, and first rows loaded from the same debiasRdata package.

4.1 Complete output inventory

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
`data_counts.csv`	5	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`input_schemas.csv`	27	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`lad_OD_travel2work.head.csv`	6	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`census_lad_OD_travel2work.head.csv`	6	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`coverage_lad.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`lad_covariates.head.csv`	6	10	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`lad_centroids.head.csv`	6	6	1×10^{-10}	1×10^{-8}	exact values	**PASS**

Figures. This vignette section is table/input oriented; the declared article audit contains no data-figure contract for it.

4.2 Selected side-by-side table values

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under `audit/goldens/r/articles` and `audit/goldens/python/articles`.

4.2.1 `data_counts.csv`

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	object	lad_OD_travel2work	lad_OD_travel2work	0	exact
1	rows	42455	42455	0	exact
1	columns	3	3	0	exact
2	object	census_lad_OD_ travel2work	census_lad_OD_ travel2work	0	exact
2	rows	69983	69983	0	exact
2	columns	3	3	0	exact
3	object	coverage_lad	coverage_lad	0	exact

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
3	rows	331	331	0	exact
3	columns	5	5	0	exact
4	object	lad_covariates	lad_covariates	0	exact
4	rows	331	331	0	exact
4	columns	10	10	0	exact
5	object	lad_centroids	lad_centroids	0	exact
5	rows	363	363	0	exact
5	columns	6	6	0	exact

Acceptance tolerance: 1×10^{-8}.

4.2.2 `coverage_lad.head.csv`

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	code	E06000001	E06000001	0	exact
1	population	92338	92338	0	exact
1	user_count	1207	1207	0	exact
2	code	E06000002	E06000002	0	exact
2	population	143926	143926	0	exact
2	user_count	1028	1028	0	exact
3	code	E06000003	E06000003	0	exact
3	population	136531	136531	0	exact
3	user_count	1144	1144	0	exact
4	code	E06000004	E06000004	0	exact
4	population	196595	196595	0	exact
4	user_count	1755	1755	0	exact
5	code	E06000005	E06000005	0	exact
5	population	107799	107799	0	exact
5	user_count	1165	1165	0	exact
6	code	E06000006	E06000006	0	exact
6	population	128478	128478	0	exact
6	user_count	1744	1744	0	exact

Acceptance tolerance: 1×10^{-8}.

4.2.3 `lad_covariates.head.csv`

R code: `check_article_parity.R`
Python code: `example_data.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	area	E06000001	E06000001	0	exact
1	per_age_20.29	11.48510348	11.48510348	0	exact
1	per_level4	24.80518437	24.80518437	0	exact
1	rural_pct	3.094915805	3.094915805	0	exact
2	area	E06000002	E06000002	0	exact
2	per_age_20.29	14.23856704	14.23856704	0	exact

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
2	per_level4	26.44134142	26.44134142	0	exact
2	rural_pct	0.2195404934	0.2195404934	0	exact
3	area	E06000003	E06000003	0	exact
3	per_age_20.29	10.28029854	10.28029854	0	exact
3	per_level4	24.92656353	24.92656353	0	exact
3	rural_pct	31.69232798	31.69232798	0	exact
4	area	E06000004	E06000004	0	exact
4	per_age_20.29	10.99061522	10.99061522	0	exact
4	per_level4	29.52218214	29.52218214	0	exact
4	rural_pct	5.552786312	5.552786312	0	exact
5	area	E06000005	E06000005	0	exact
5	per_age_20.29	10.9507333	10.9507333	0	exact
5	per_level4	28.96137651	28.96137651	0	exact
5	rural_pct	13.05569726	13.05569726	0	exact
6	area	E06000006	E06000006	0	exact
6	per_age_20.29	11.41587341	11.41587341	0	exact
6	per_level4	23.93119637	23.93119637	0	exact
6	rural_pct	0.7223307621	0.7223307621	0	exact

Acceptance tolerance: 1×10^{-8}.

5 Measuring Coverage Bias

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v04-measuring-coverage-bias.html

Checks coverage-bias tables, total coverage calculations, distribution summaries, residual-bias tables, and plot data contracts.

5.1 Complete output inventory

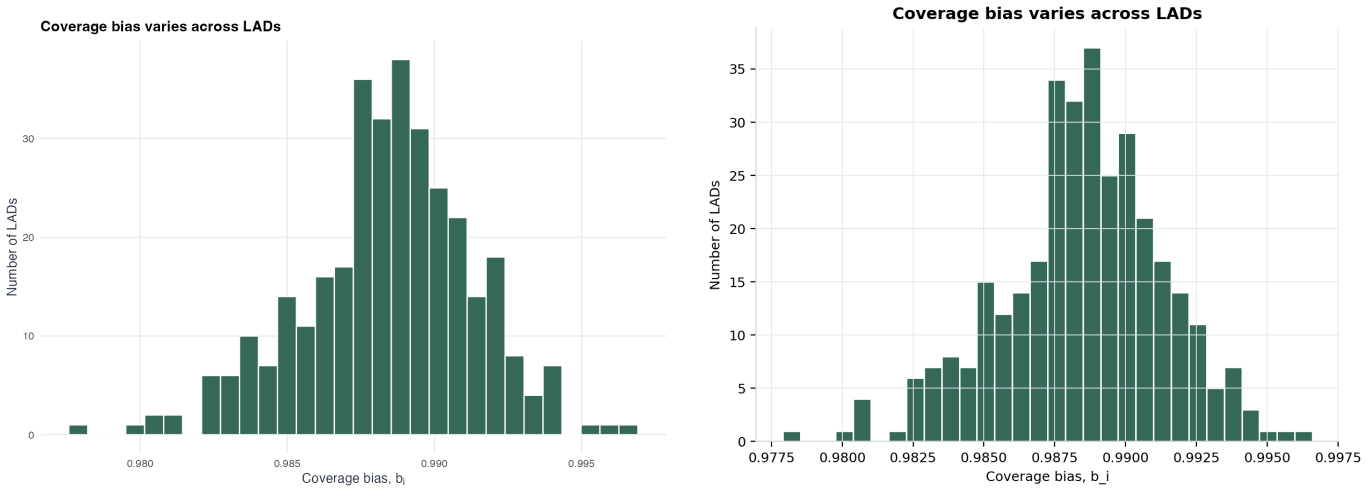
R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
`coverage_lad.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`coverage_renamed.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`measure_bias.head.csv`	6	6	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`coverage_totals.csv`	1	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`lowest_bias.head.csv`	6	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`distribution.summary.csv`	1	11	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`distribution.top_share_diff.csv`	6	12	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`residual.summary.csv`	1	14	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`residual.definitions.csv`	4	3	–	1×10^{-8}	exact values	**PASS**
`residual.area_head.csv`	6	7	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`plot_contracts.csv`	4	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**

5.2 Rendered figure pairs

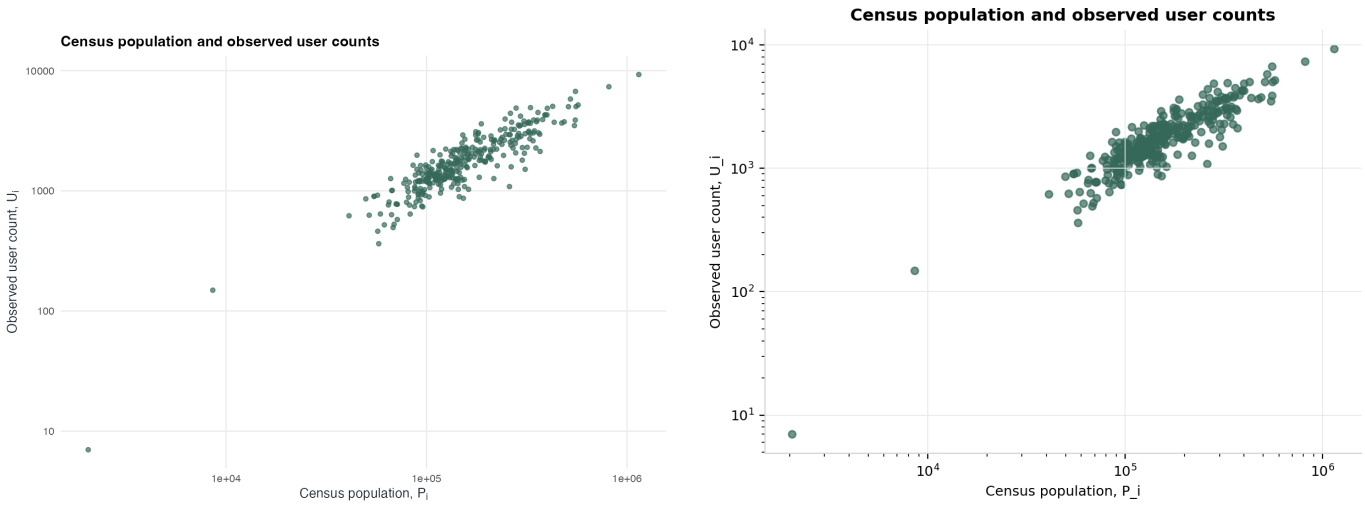
R code: `render_vignette_figures.R`
Python code: `render_vignette_latex_comparison.py`
Test: `check_article_parity.py`; `test_validation_plot_contract.py`

Each pair uses the corresponding full-data or committed-summary R and Python values. Visual style differences are expected because the backends are ggplot2 and Matplotlib.



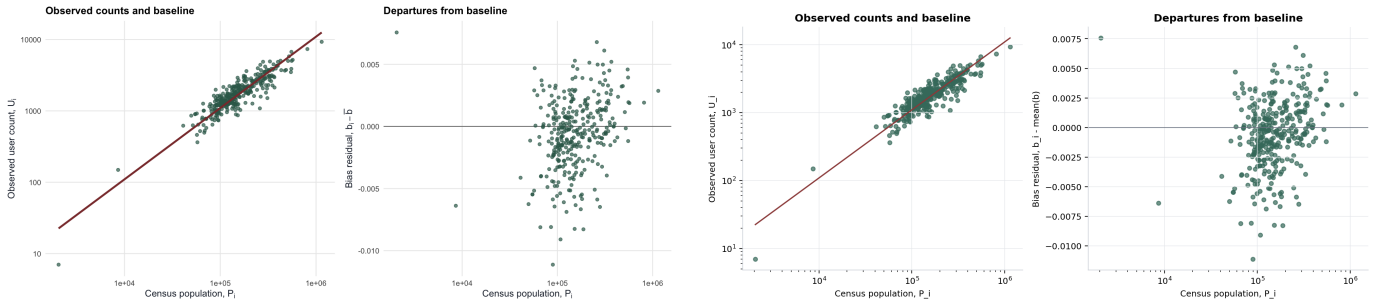
(a) R / debiasR (b) Python / debiaspy

Figure 1: Coverage-bias histogram. Full 331-area data; the same variable, bins and interpretation are used.



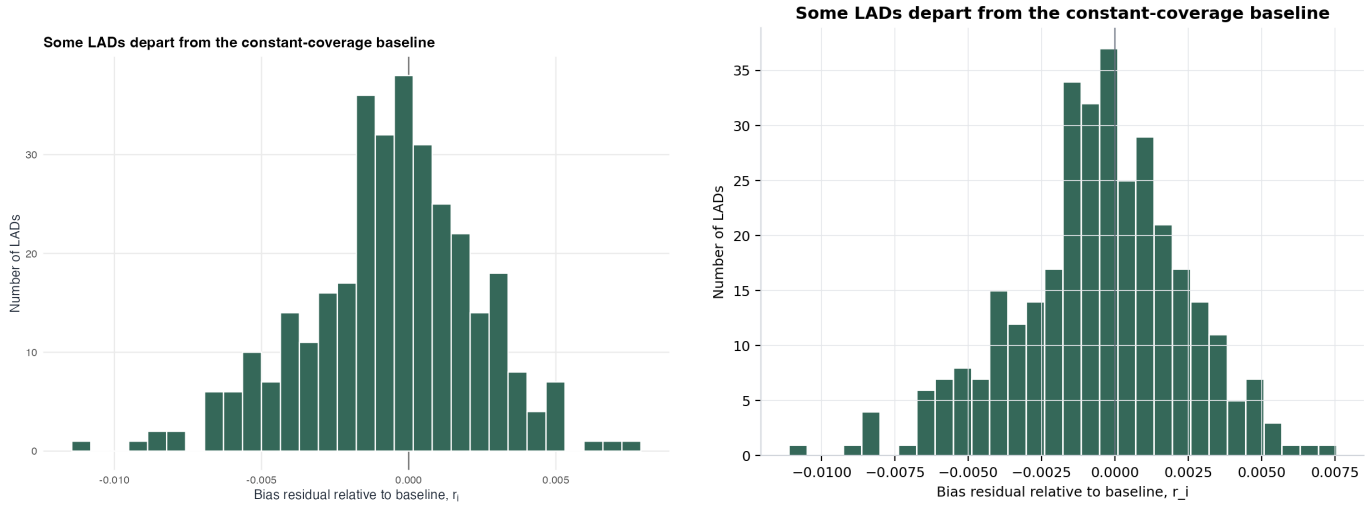
(a) R / debiasR (b) Python / debiaspy

Figure 2: Population and observed-user scatter. Full 331-area data with logarithmic axes.



(a) R / debiasR (b) Python / debiaspy

Figure 3: Constant-coverage baseline and residual panel. Observed counts, expected baseline and residual bias use the same formulas.



(a) R / debiasR (b) Python / debiaspy

Figure 4: Residual-bias histogram. Full 331-area residual table; rendered pixels are backend-specific.

5.3 Figure-data contracts

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

The committed contract rows agree exactly.

No.	R plot	R source/axes	Python plot	Python source/axes	Result
1	coverage_bias_ histogram	x=coverage_bias; y=count; n=331	coverage_bias_ histogram	x=coverage_bias; y=count; n=331	**PASS**
2	population_user_ count_scatter	x=population; y=user_count; n=331	population_user_ count_scatter	x=population; y=user_count; n=331	**PASS**
3	baseline_residual_ panel	x=population; y=residual_ bias; n=331	baseline_residual_ panel	x=population; y=residual_ bias; n=331	**PASS**
4	residual_bias_ histogram	x=residual_bias; y=count; n=331	residual_bias_ histogram	x=residual_bias; y=count; n=331	**PASS**

5.4 Selected side-by-side table values

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under `audit/goldens/r/articles` and `audit/goldens/python/articles`.

5.4.1 `coverage_totals.csv`

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	lad_count	331	331	0	exact
1	census_pop	59597541	59597541	0	exact
1	digital_trace_pop	654130.2188	654130.2188	0	exact
1	overall_coverage	0.01097579208	0.01097579208	0	exact
1	overall_coverage_bias	0.9890242079	0.9890242079	0	exact

Acceptance tolerance: 1×10^{-8}.

5.4.2 `measure_bias.head.csv`

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	area	E06000001	E06000001	0	exact
1	population	92338	92338	0	exact
1	user_count	1207	1207	0	exact
1	coverage_score	0.01307154151	0.01307154151	0	exact
1	coverage_bias	0.9869284585	0.9869284585	0	exact
2	area	E06000002	E06000002	0	exact
2	population	143926	143926	0	exact
2	user_count	1028	1028	0	exact
2	coverage_score	0.007142559371	0.007142559371	0	exact
2	coverage_bias	0.9928574406	0.9928574406	0	exact
3	area	E06000003	E06000003	0	exact
3	population	136531	136531	0	exact
3	user_count	1144	1144	0	exact
3	coverage_score	0.008379049447	0.008379049447	0	exact
3	coverage_bias	0.9916209506	0.9916209506	0	exact
4	area	E06000004	E06000004	0	exact
4	population	196595	196595	0	exact
4	user_count	1755	1755	0	exact
4	coverage_score	0.008926981866	0.008926981866	0	exact
4	coverage_bias	0.9910730181	0.9910730181	0	exact
5	area	E06000005	E06000005	0	exact
5	population	107799	107799	0	exact
5	user_count	1165	1165	0	exact
5	coverage_score	0.01080715034	0.01080715034	0	exact
5	coverage_bias	0.9891928497	0.9891928497	0	exact
6	area	E06000006	E06000006	0	exact
6	population	128478	128478	0	exact
6	user_count	1744	1744	0	exact
6	coverage_score	0.01357430844	0.01357430844	0	exact

Row	Field	R output	Python output	$\|\Delta\|$	Result
6	coverage_bias	0.9864256916	0.9864256916	0	exact

Acceptance tolerance: 1×10^{-8}.

5.4.3 `distribution.summary.csv`

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	n_areas	331	331	0	exact
1	kl_population_user	0.03321172694	0.03321172694	0	exact
1	jsd_population_user	0.008103762933	0.008103762933	0	exact
1	mean_abs_share_difference	0.0005957367869	0.0005957367869	0	exact
1	max_abs_share_difference	0.005013364012	0.005013364012	0	exact

Acceptance tolerance: 1×10^{-8}.

5.4.4 `residual.summary.csv`

R code: `measure_bias.R`; `distribution_metrics.R`
Python code: `measure_bias.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_measure_bias_r_parity.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	n_areas	331	331	0	exact
1	global_coverage_score	0.01097579208	0.01097579208	0	exact
1	mean_selected_residual	0.0005838313134	0.0005838313134	0	exact
1	sd_selected_residual	0.00291139526	0.00291139526	0	exact
1	moran_i	NA	NA	0	exact

Acceptance tolerance: 1×10^{-8}.

6 Identifying And Explaining Bias

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v05-identifying-and-explaining-bias.html

Checks the same LAD coverage/covariate join and the residual-bias OLS coefficient table used by the article explanation workflow.

6.1 Complete output inventory

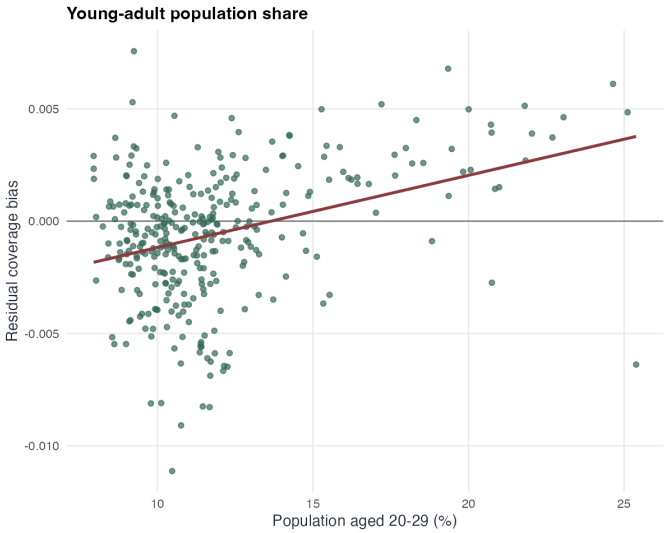
R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
`coverage.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`covariates.head.csv`	6	10	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`bias.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`residual.head.csv`	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`bias_covariates.head.csv`	6	6	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`coefficient_table.csv`	7	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`fitted_residual.head.csv`	6	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`top_unexplained.csv`	10	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`plot_contracts.csv`	3	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**

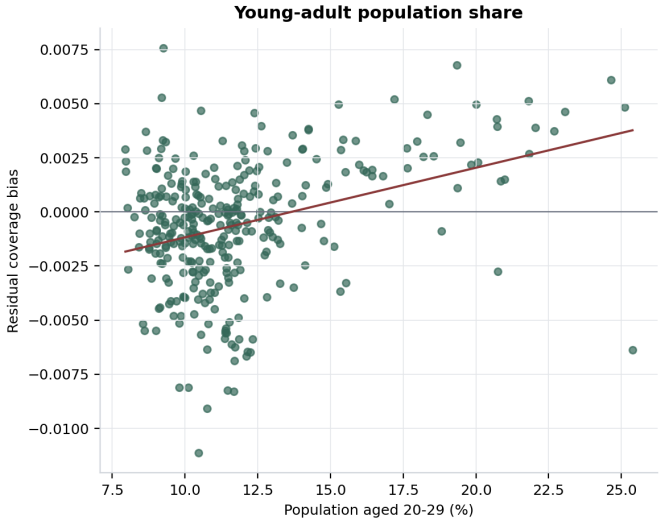
6.2 Rendered figure pairs

R code: `render_vignette_figures.R`
Python code: `render_vignette_latex_comparison.py`
Test: `check_article_parity.py`; `test_validation_plot_contract.py`

Each pair uses the corresponding full-data or committed-summary R and Python values. Visual style differences are expected because the backends are ggplot2 and Matplotlib.



(a) R / debiasR



(b) Python / debiaspy

Figure 5: Residual bias and young-adult share. Same full joined data and least-squares trend semantics.

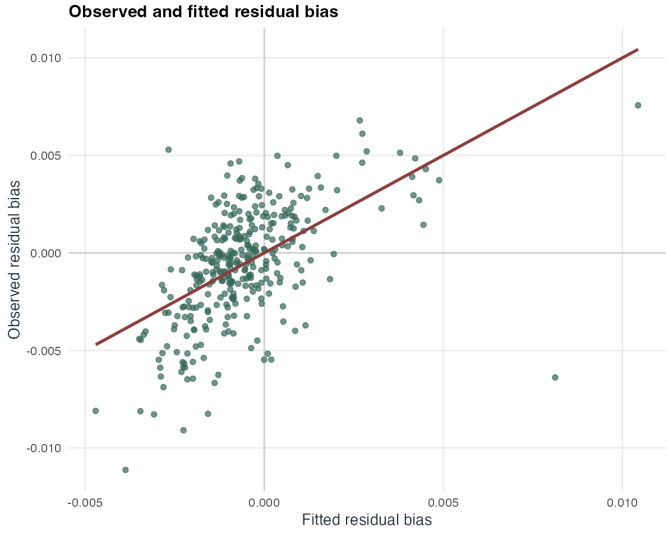


(a) R / debiasR

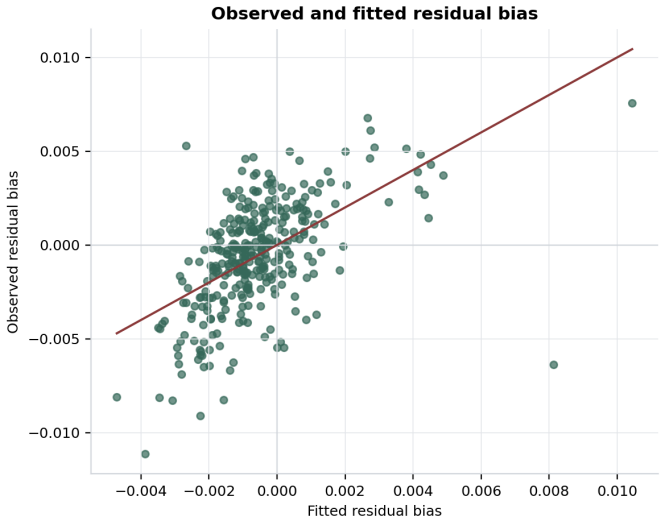


(b) Python / debiaspy

Figure 6: Residual bias and level-4 qualification share. Same full joined data and least-squares trend semantics.



(a) R / debiasR



(b) Python / debiaspy

Figure 7: Observed and fitted residual bias. Same six-variable OLS model and fitted values.

6.3 Figure-data contracts

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

The committed contract rows agree exactly.

No.	R plot	R source/axes	Python plot	Python source/axes	Result
1	residual_vs_age_20_29	x=per_age_20.29; y=residual_bias; n=331	residual_vs_age_20_29	x=per_age_20.29; y=residual_bias; n=331	**PASS**
2	residual_vs_level4	x=per_level4; y=residual_bias; n=331	residual_vs_level4	x=per_level4; y=residual_bias; n=331	**PASS**
3	observed_vs_fitted_residual	x=fitted_residual_bias; y=residual_bias; n=331	observed_vs_fitted_residual	x=fitted_residual_bias; y=residual_bias; n=331	**PASS**

6.4 Selected side-by-side table values

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under `audit/goldens/r/articles` and `audit/goldens/python/articles`.

6.4.1 `coefficient_table.csv`

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	term	(Intercept)	(Intercept)	0	exact
1	estimate	-0.02133357703	-0.02133357703	0	exact
1	std_error	0.002558308434	0.002558308434	0	exact
1	statistic	-8.338938633	-8.338938633	0	exact
1	p_value	0	0	0	exact
2	term	per_age_20.29	per_age_20.29	0	exact
2	estimate	0.0002300285588	0.0002300285588	0	exact
2	std_error	6.736613209e-05	6.736613209e-05	0	exact
2	statistic	3.414602436	3.414602436	0	exact
2	p_value	0.0007199557938	0.0007199557938	0	exact
3	term	per_age_70plus	per_age_70plus	0	exact
3	estimate	0.000215469174	0.000215469174	0	exact
3	std_error	6.687210769e-05	6.687210769e-05	0	exact
3	statistic	3.222108311	3.222108311	0	exact
3	p_value	0.001401819885	0.001401819885	0	exact
4	term	per_level4	per_level4	0	exact
4	estimate	0.0002347166805	0.0002347166805	0	exact
4	std_error	3.47032463e-05	3.47032463e-05	0	exact
4	statistic	6.763536714	6.763536714	0	exact
4	p_value	6.297558623e-11	6.297558623e-11	0	exact
5	term	per_hh_no_centralheat	per_hh_no_centralheat	0	exact
5	estimate	0.0006680707387	0.0006680707387	0	exact
5	std_error	0.0001332254099	0.0001332254099	0	exact
5	statistic	5.014589478	5.014589478	0	exact
5	p_value	8.769748953e-07	8.769748953e-07	0	exact
6	term	per_NS_SeC_L13_routine	per_NS_SeC_L13_routine	0	exact
6	estimate	0.0005217525003	0.0005217525003	0	exact
6	std_error	7.486136936e-05	7.486136936e-05	0	exact
6	statistic	6.969582639	6.969582639	0	exact
6	p_value	1.786132126e-11	1.786132126e-11	0	exact
7	term	rural_pct	rural_pct	0	exact
7	estimate	-7.687229237e-06	-7.687229237e-06	0	exact
7	std_error	8.213160523e-06	8.213160523e-06	0	exact
7	statistic	-0.935964811	-0.935964811	0	exact
7	p_value	0.3499883927	0.3499883927	0	exact

Acceptance tolerance: 1×10^{-8}.

6.4.2 `fitted_residual.head.csv`

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	area	E06000001	E06000001	0	exact
1	residual_bias	-0.002095749426	-0.002095749426	0	exact
1	fitted_residual_bias	-0.001104654496	-0.001104654496	0	exact
1	unexplained_residual	-0.0009910949295	-0.0009910949295	0	exact
2	area	E06000002	E06000002	0	exact
2	residual_bias	0.003833232714	0.003833232714	0	exact
2	fitted_residual_bias	-0.000621217643	-0.000621217643	0	exact
2	unexplained_residual	0.004454450357	0.004454450357	0	exact
3	area	E06000003	E06000003	0	exact
3	residual_bias	0.002596742638	0.002596742638	0	exact
3	fitted_residual_bias	-0.00127889273	-0.00127889273	0	exact
3	unexplained_residual	0.003875635368	0.003875635368	0	exact
4	area	E06000004	E06000004	0	exact
4	residual_bias	0.002048810219	0.002048810219	0	exact
4	fitted_residual_bias	-0.001107056816	-0.001107056816	0	exact
4	unexplained_residual	0.003155867035	0.003155867035	0	exact
5	area	E06000005	E06000005	0	exact
5	residual_bias	0.0001686417403	0.0001686417403	0	exact
5	fitted_residual_bias	-0.0003316001714	-0.0003316001714	0	exact
5	unexplained_residual	0.0005002419117	0.0005002419117	0	exact
6	area	E06000006	E06000006	0	exact
6	residual_bias	-0.002598516357	-0.002598516357	0	exact
6	fitted_residual_bias	-0.00080466704	-0.00080466704	0	exact
6	unexplained_residual	-0.001793849317	-0.001793849317	0	exact

Acceptance tolerance: 1×10^{-8}.

6.4.3 `top_unexplained.csv`

R code: `measure_bias.R`; `check_article_parity.R`
Python code: `measure_bias.py`; `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	area	E09000001	E09000001	0	exact
1	name	City of London	City of London	0	exact
1	residual_bias	-0.006384105387	-0.006384105387	0	exact
1	fitted_residual_bias	0.008122286251	0.008122286251	0	exact
1	unexplained_residual	-0.01450639164	-0.01450639164	0	exact
2	area	E07000228	E07000228	0	exact
2	name	Mid Sussex	Mid Sussex	0	exact
2	residual_bias	0.005293005619	0.005293005619	0	exact
2	fitted_residual_bias	-0.002672141321	-0.002672141321	0	exact
2	unexplained_residual	0.007965146939	0.007965146939	0	exact
3	area	E07000069	E07000069	0	exact
3	name	Castle Point	Castle Point	0	exact
3	residual_bias	-0.01112562888	-0.01112562888	0	exact
3	fitted_residual_bias	-0.003874755351	-0.003874755351	0	exact

Row	Field	R output	Python output	|Δ|	Result
3	unexplained_residual	-0.007250873529	-0.007250873529	0	exact
4	area	E07000098	E07000098	0	exact
4	name	Hertsmere	Hertsmere	0	exact
4	residual_bias	-0.009093392813	-0.009093392813	0	exact
4	fitted_residual_bias	-0.002256288912	-0.002256288912	0	exact
4	unexplained_residual	-0.006837103901	-0.006837103901	0	exact
5	area	E07000113	E07000113	0	exact
5	name	Swale	Swale	0	exact
5	residual_bias	-0.008249398453	-0.008249398453	0	exact
5	fitted_residual_bias	-0.001571936489	-0.001571936489	0	exact
5	unexplained_residual	-0.006677461963	-0.006677461963	0	exact
6	area	E07000030	E07000030	0	exact
6	name	Eden	Eden	0	exact
6	residual_bias	-0.005470657443	-0.005470657443	0	exact
6	fitted_residual_bias	0.0001957089225	0.0001957089225	0	exact
6	unexplained_residual	-0.005666366365	-0.005666366365	0	exact
7	area	E08000032	E08000032	0	exact
7	name	Bradford	Bradford	0	exact
7	residual_bias	0.004585010038	0.004585010038	0	exact
7	fitted_residual_bias	-0.0009408765204	-0.0009408765204	0	exact
7	unexplained_residual	0.005525886558	0.005525886558	0	exact
8	area	E07000167	E07000167	0	exact
8	name	Ryedale	Ryedale	0	exact
8	residual_bias	-0.005474680365	-0.005474680365	0	exact
8	fitted_residual_bias	-4.322944617e-06	-4.322944617e-06	0	exact
8	unexplained_residual	-0.00547035742	-0.00547035742	0	exact
9	area	E07000135	E07000135	0	exact
9	name	Oadby and Wigston	Oadby and Wigston	0	exact
9	residual_bias	0.004689751252	0.004689751252	0	exact
9	fitted_residual_bias	-0.0007009293555	-0.0007009293555	0	exact
9	unexplained_residual	0.005390680607	0.005390680607	0	exact
10	area	E06000034	E06000034	0	exact
10	name	Thurrock	Thurrock	0	exact
10	residual_bias	-0.006666153131	-0.006666153131	0	exact
10	fitted_residual_bias	-0.001387336853	-0.001387336853	0	exact
10	unexplained_residual	-0.005278816279	-0.005278816279	0	exact

Acceptance tolerance: 1×10^{-8}.

7 Adjusting Biases

R code: `adjust_all_methods.R`; `adjust_inverse_penetration.R`; `adjust_raking_ratio.R`; `adjust_coefficient.R`; `adjust_selection_rate.R`; `adjust_selection_rate2.R`; `adjust_multilevel_bayes.R`
Python code: `adjust_all_methods.py`; `adjust_inverse_penetration.py`; `adjust_raking_ratio.py`; `adjust_coefficient.py`; `adjust_selection_rate.py`; `adjust_selection_rate2.py`; `adjust_multilevel_bayes.py`
Test: `check_article_parity.py`; `test_r_adjustment_parity.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v06-adjusting-biases.html

Checks algebraic adjustment previews, calibrated attributes, raking summaries, maximum-likelihood adjust_all_methods output, and ingestion of precomputed Bayesian article files.

7.1 Complete output inventory

R code: adjust_all_methods.R; adjust_inverse_penetration.R; adjust_raking_ratio.R; adjust_coefficient.R; adjust_selection_rate.R; adjust_selection_rate2.R; adjust_multilevel_bayes.R
Python code: adjust_all_methods.py; adjust_inverse_penetration.py; adjust_raking_ratio.py; adjust_coefficient.py; adjust_selection_rate.py; adjust_selection_rate2.py; adjust_multilevel_bayes.py
Test: check_article_parity.py; test_r_adjustment_parity.py

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
raw_heads.OD_travel2work.csv	6	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
raw_heads.census_OD_travel2work.csv	6	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
raw_heads.coverage.csv	6	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
example_counts.csv	5	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
inverse_penetration.preview.csv	5	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
selection_rate.preview.csv	5	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
selection_rate.attrs.csv	1	2	1×10^{-10}	1×10^{-8}	exact values	**PASS**
selection_rate2.preview.csv	5	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
selection_rate2.attrs.csv	1	2	1×10^{-10}	1×10^{-8}	exact values	**PASS**
raking_ratio.preview.csv	5	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
raking_ratio.summary.csv	1	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
coefficient.preview.csv	5	5	1×10^{-10}	1×10^{-8}	exact values	**PASS**
coefficient.attrs.csv	1	2	1×10^{-10}	1×10^{-8}	exact values	**PASS**
bayesian_precomputed.display.csv	5	9	1×10^{-10}	1×10^{-8}	exact values	**PASS**
bayesian _ precomputed . metadata . csv	12	2	–	1×10^{-8}	exact values	**PASS**
bayesian_precomputed.model_terms.csv	16	2	–	1×10^{-8}	exact values	**PASS**
adjust_all_methods.summary.csv	6	4	1×10^{-6}	1×10^{-6}	exact values	**PASS**

Figures. This vignette section is table/input oriented; the declared article audit contains no data-figure contract for it.

7.2 Selected side-by-side table values

R code: adjust_all_methods.R; adjust_inverse_penetration.R; adjust_raking_ratio.R; adjust_coefficient.R; adjust_selection_rate.R; adjust_selection_rate2.R; adjust_multilevel_bayes.R
Python code: adjust_all_methods.py; adjust_inverse_penetration.py; adjust_raking_ratio.py; adjust_coefficient.py; adjust_selection_rate.py; adjust_selection_rate2.py; adjust_multilevel_bayes.py
Test: check_article_parity.py; test_r_adjustment_parity.py

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under `audit/goldens/r/articles` and `audit/goldens/python/articles`.

7.2.1 inverse_penetration.preview.csv

R code: adjust_inverse_penetration.R
Python code: adjust_inverse_penetration.py
Test: check_article_parity.py; test_r_adjustment_parity.py

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	origin	E06000011	E06000011	0	exact
1	destination	E06000011	E06000011	0	exact
1	flow_mpd_raw	2403	2403	0	exact
1	flow_mpd_adjusted	53757.61086	53757.61086	0	exact
1	flow_benchmark	57155	57155	0	exact
2	origin	E06000011	E06000011	0	exact
2	destination	E06000016	E06000016	0	exact
2	flow_mpd_raw	1	1	0	exact
2	flow_mpd_adjusted	22.37104072	22.37104072	0	exact
2	flow_benchmark	10	10	0	exact

Row	Field	R output	Python output	|Δ|	Result
3	origin	E06000011	E06000011	0	exact
3	destination	E06000018	E06000018	0	exact
3	flow_mpd_raw	1	1	0	exact
3	flow_mpd_adjusted	22.37104072	22.37104072	0	exact
3	flow_benchmark	15	15	0	exact
4	origin	E06000011	E06000011	0	exact
4	destination	E06000023	E06000023	0	exact
4	flow_mpd_raw	2	2	0	exact
4	flow_mpd_adjusted	44.74208145	44.74208145	0	exact
4	flow_benchmark	3	3	0	exact
5	origin	E06000011	E06000011	0	exact
5	destination	E06000047	E06000047	0	exact
5	flow_mpd_raw	6	6	0	exact
5	flow_mpd_adjusted	134.2262443	134.2262443	0	exact
5	flow_benchmark	11	11	0	exact

Acceptance tolerance: 1×10^{-8}.

7.2.2 `raking_ratio.summary.csv`

R code: `adjust_raking_ratio.R`
Python code: `adjust_raking_ratio.py`
Test: `check_article_parity.py`; `test_r_adjustment_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	raw_total	80695	80695	0	exact
1	adjusted_total	2302435	2302435	0	exact
1	median_weight	27.8627201	27.8627201	0	exact
1	ipf_converged	True	True	0	exact
1	ipf_iterations	172	172	0	exact

Acceptance tolerance: 1×10^{-8}.

7.2.3 `adjust_all_methods.summary.csv`

R code: `adjust_all_methods.R`; `adjust_multilevel_bayes.R`
Python code: `adjust_all_methods.py`; `adjust_multilevel_bayes.py`
Test: `check_article_parity.py`; `test_r_adjustment_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	method	inverse_penetration	inverse_penetration	0	exact
1	engine	direct_adjustment	direct_adjustment	0	exact
1	rows	625	625	0	exact
1	adjusted_total	2299443.401	2299443.401	0	exact
2	method	selection_rate	selection_rate	0	exact
2	engine	direct_adjustment	direct_adjustment	0	exact
2	rows	625	625	0	exact
2	adjusted_total	2055645.469	2055645.469	0	exact
3	method	selection_rate2	selection_rate2	0	exact
3	engine	direct_adjustment	direct_adjustment	0	exact
3	rows	625	625	0	exact
3	adjusted_total	76930.66889	76930.66889	0	exact

Row	Field	R output	Python output	|Δ|	Result
4	method	raking_ratio	raking_ratio	0	exact
4	engine	direct_adjustment	direct_adjustment	0	exact
4	rows	625	625	0	exact
4	adjusted_total	2302435	2302435	0	exact
5	method	coefficient	coefficient	0	exact
5	engine	direct_adjustment	direct_adjustment	0	exact
5	rows	625	625	0	exact
5	adjusted_total	2331903.475	2331903.475	0	exact
6	method	multilevel_bayes	multilevel_bayes	0	exact
6	engine	frequentist	frequentist	0	exact
6	rows	625	625	0	exact
6	adjusted_total	56234.83351	56234.83351	0	exact

Acceptance tolerance: 1×10^{-6}.

7.2.4 `bayesian_precomputed.display.csv`

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	origin	E06000011	E06000011	0	exact
1	destination	E06000011	E06000011	0	exact
1	flow	2403	2403	0	exact
1	flow_adj	82289.892	82289.892	0	exact
1	flow_benchmark	57155	57155	0	exact
2	origin	E06000011	E06000011	0	exact
2	destination	E06000016	E06000016	0	exact
2	flow	1	1	0	exact
2	flow_adj	506.2006562	506.2006562	0	exact
2	flow_benchmark	10	10	0	exact
3	origin	E06000011	E06000011	0	exact
3	destination	E06000018	E06000018	0	exact
3	flow	1	1	0	exact
3	flow_adj	523.6378868	523.6378868	0	exact
3	flow_benchmark	15	15	0	exact
4	origin	E06000011	E06000011	0	exact
4	destination	E06000023	E06000023	0	exact
4	flow	2	2	0	exact
4	flow_adj	461.6358699	461.6358699	0	exact
4	flow_benchmark	3	3	0	exact
5	origin	E06000011	E06000011	0	exact
5	destination	E06000047	E06000047	0	exact
5	flow	6	6	0	exact
5	flow_adj	508.6053287	508.6053287	0	exact
5	flow_benchmark	11	11	0	exact

Acceptance tolerance: 1×10^{-8}.

Bayesian boundary for v06. The article harness loads the same versioned precomputed Bayesian files in R and Python. Downstream v07 validation is rerun on those rows. Independent model fitting is demonstrated separately by the controlled 12-case Bayesian audit, not by these vignette files.

8 Validation

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v07-validation.html

Checks full-LAD validation calls executed without MCMC plus ingestion of precomputed Bayesian files. Local Moran permutation outputs and rendered map pixels are not compared.

8.1 Complete output inventory

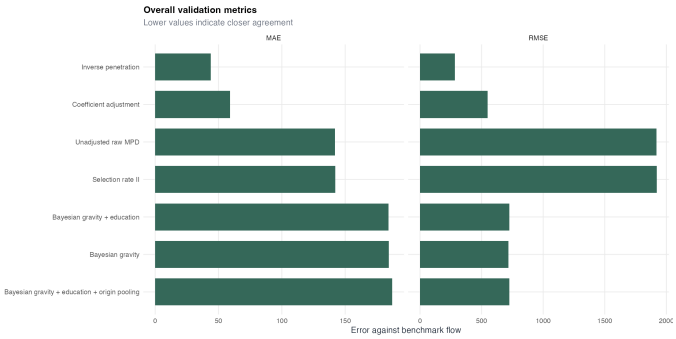
R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
`example_counts.csv`	5	3	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`method_set.csv`	7	3	–	1×10^{-6}	exact values	**PASS**
`bayesian_precomputed.display.csv`	35	30	1×10^{-6}	1×10^{-6}	exact values	**PASS**
`bayesian _ precomputed . metadata . csv`	7	64	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`level1.overall_metrics.csv`	7	21	1×10^{-5}	1×10^{-5}	exact values	**PASS**
`level2.origin_margin_metrics.csv`	4	21	2×10^{-4}	2×10^{-4}	exact values	**PASS**
`level2 . destination _ margin _ metrics.csv`	4	21	2×10^{-4}	2×10^{-4}	exact values	**PASS**
`level3.residual_summary.csv`	4	21	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`level3.top_worst.csv`	20	41	1×10^{-6}	1×10^{-6}	exact values	**PASS**
`level4.distribution_summary.csv`	4	16	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`level4.origin_head.csv`	20	23	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`level5.neighbor_summary.csv`	1	4	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`level5.structure_summary.csv`	4	11	1×10^{-10}	1×10^{-6}	exact values	**PASS**
`plot_contracts.csv`	7	3	–	1×10^{-6}	exact values	**PASS**

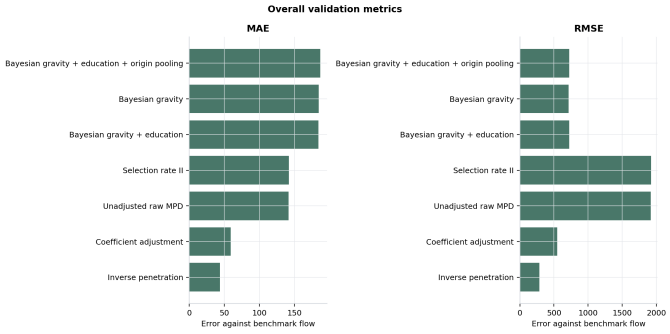
8.2 Rendered figure pairs

R code: `render_vignette_figures.R`
Python code: `render_vignette_latex_comparison.py`
Test: `check_article_parity.py`; `test_validation_plot_contract.py`

Each pair uses the corresponding full-data or committed-summary R and Python values. Visual style differences are expected because the backends are ggplot2 and Matplotlib.

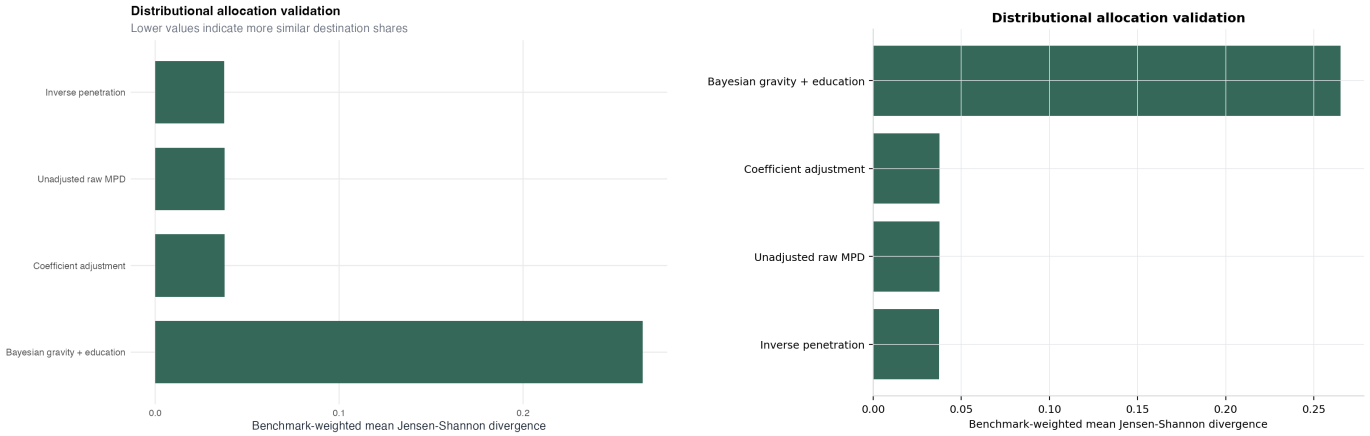


(a) R / debiasR



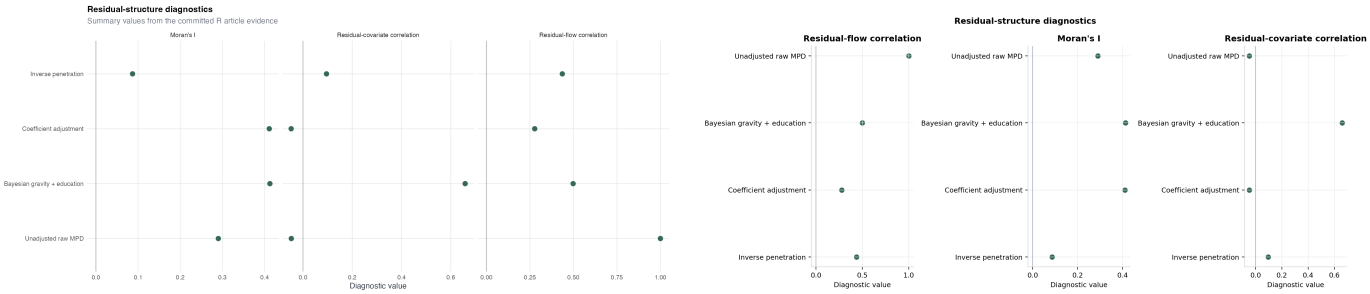
(b) Python / debiaspy

Figure 8: Level 1 overall validation metrics. Summary rendering from the committed seven-method output table.



(a) R / debiasR (b) Python / debiaspy

Figure 9: Level 4 distributional allocation. Summary rendering from the committed four-method distribution table.



(a) R / debiasR (b) Python / debiaspy

Figure 10: Level 5 residual structure. Summary rendering from the committed residual-structure table.

8.3 Figure-data contracts

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

The committed contract rows agree exactly. Four additional vignette visuals (origin scatter, destination scatter, flow scatter and residual bands) require full residual rows that are not committed as article goldens. The optional LISA map also requires seeded permutations and external boundary data. They remain contract-only here.

No.	R plot	R source/axes	Python plot	Python source/axes	Result
1	level1_metrics	level1.overall_metrics	level1_metrics	level1.overall_metrics	**PASS**
2	level2_origin_scatter	origin_margin_residuals	level2_origin_scatter	origin_margin_residuals	**PASS**
3	level2_destination_ scatter	destination_margin_ residuals	level2_destination_ scatter	destination_margin_ residuals	**PASS**
4	level3_flow_scatter	level3.residual_summary	level3_flow_scatter	level3.residual_summary	**PASS**
5	level3_residual_ bands	level3.residual_summary	level3_residual_ bands	level3.residual_summary	**PASS**
6	level4_distribution_ pairwise	level4.distribution_summary	level4_distribution_ pairwise	level4.distribution_summary	**PASS**
7	level5_structure	level5.structure_summary	level5_structure	level5.structure_summary	**PASS**

8.4 Selected side-by-side table values

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under `audit/goldens/r/articles` and `audit/goldens/python/articles`.

8.4.1 `level1.overall_metrics.csv`

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	method	unadjusted	unadjusted	0	exact
1	n	97969	97969	0	exact
1	mae	141.8334984	141.8334984	0	exact
1	rmse	1920.035703	1920.035703	0	exact
1	pearson_r	0.9611769098	0.9611769098	0	exact
1	spearman_rho	0.5385132802	0.5385132802	0	exact
2	method	inverse_penetration	inverse_penetration	0	exact
2	n	97969	97969	0	exact
2	mae	43.93677566	43.93677566	0	exact
2	rmse	284.3969114	284.3969114	0	exact
2	pearson_r	0.990717465	0.990717465	0	exact
2	spearman_rho	0.5374806252	0.5374806252	0	exact
3	method	selection_rate2	selection_rate2	0	exact
3	n	97969	97969	0	exact
3	mae	142.1112575	142.1112575	0	exact
3	rmse	1923.494913	1923.494913	0	exact
3	pearson_r	0.9610399519	0.9610399519	0	exact
3	spearman_rho	0.5303185939	0.5303185939	0	exact
4	method	coefficient	coefficient	0	exact
4	n	97969	97969	0	exact
4	mae	59.13678079	59.13678079	0	exact
4	rmse	549.0047229	549.0047229	0	exact
4	pearson_r	0.9611769098	0.9611769098	0	exact
4	spearman_rho	0.5385132802	0.5385132802	0	exact
5	method	bayes_gravity	bayes_gravity	0	exact
5	n	97969	97969	0	exact
5	mae	184.4710066	184.4710066	0	exact
5	rmse	716.8306514	716.8306514	0	exact
5	pearson_r	0.9351067734	0.9351067734	0	exact
5	spearman_rho	0.4600435335	0.4600435335	0	exact
6	method	bayes_gravity_education	bayes_gravity_education	0	exact
6	n	97969	97969	0	exact
6	mae	184.0999411	184.0999411	0	exact
6	rmse	724.3472081	724.3472081	0	exact
6	pearson_r	0.9335139104	0.9335139104	0	exact
6	spearman_rho	0.4758617005	0.4758617005	0	exact
7	method	bayes_origin_pool	bayes_origin_pool	0	exact
7	n	97969	97969	0	exact
7	mae	187.0241396	187.0241396	0	exact
7	rmse	725.5390412	725.5390412	0	exact
7	pearson_r	0.9331118873	0.9331118873	0	exact
7	spearman_rho	0.4762769897	0.4762769897	0	exact

Acceptance tolerance: 1×10^{-5}.

8.4.2 `level2.origin_margin_metrics.csv`

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	method	inverse_penetration	inverse_penetration	0	exact
1	n	313	313	0	exact
1	mae	1836.217696	1836.217696	0	exact
1	rmse	2695.92716	2695.92716	0	exact
1	pearson_r	0.9962508076	0.9962508076	0	exact
2	method	coefficient	coefficient	0	exact
2	n	313	313	0	exact
2	mae	10186.93803	10186.93803	0	exact
2	rmse	14171.92813	14171.92813	0	exact
2	pearson_r	0.8925456159	0.8925456159	0	exact
3	method	bayes_gravity_education	bayes_gravity_education	0	exact
3	n	313	313	0	exact
3	mae	18736.06786	18736.06786	0	exact
3	rmse	19320.8748	19320.8748	0	exact
3	pearson_r	0.992775392	0.992775392	0	exact
4	method	unadjusted	unadjusted	0	exact
4	n	313	313	0	exact
4	mae	44293.52716	44293.52716	0	exact
4	rmse	53025.97729	53025.97729	0	exact
4	pearson_r	0.8925456159	0.8925456159	0	exact

Acceptance tolerance: 2×10^{-4}.

8.4.3 `level3.residual_summary.csv`

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	method	inverse_penetration	inverse_penetration	0	exact
1	n	97969	97969	0	exact
1	mean_abs_residual_ reduction	97.89672269	97.89672269	0	exact
1	mean_abs_residual_mpd_ benchmark	141.8334984	141.8334984	0	exact
1	mean_abs_residual_adj_ benchmark	43.93677566	43.93677566	0	exact
1	share_improved	0.09595892578	0.09595892578	0	exact
2	method	coefficient	coefficient	0	exact
2	n	97969	97969	0	exact
2	mean_abs_residual_ reduction	82.69671756	82.69671756	0	exact
2	mean_abs_residual_mpd_ benchmark	141.8334984	141.8334984	0	exact
2	mean_abs_residual_adj_ benchmark	59.13678079	59.13678079	0	exact
2	share_improved	0.08752768733	0.08752768733	0	exact
3	method	bayes_gravity_education	bayes_gravity_education	0	exact

Row	Field	R output	Python output	|Δ|	Result
3	n	97969	97969	0	exact
3	mean_abs_residual_ reduction	-42.26644278	-42.26644278	0	exact
3	mean_abs_residual_mpd_ benchmark	141.8334984	141.8334984	0	exact
3	mean_abs_residual_adj_ benchmark	184.0999411	184.0999411	0	exact
3	share_improved	0.06193795997	0.06193795997	0	exact
4	method	unadjusted	unadjusted	0	exact
4	n	97969	97969	0	exact
4	mean_abs_residual_ reduction	0	0	0	exact
4	mean_abs_residual_mpd_ benchmark	141.8334984	141.8334984	0	exact
4	mean_abs_residual_adj_ benchmark	141.8334984	141.8334984	0	exact
4	share_improved	0	0	0	exact

Acceptance tolerance: 1×10^{-6}.

8.4.4 `level4.distribution_summary.csv`

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Row	Field	R output	Python output	|Δ|	Result
1	method	inverse_penetration	inverse_penetration	0	exact
1	n_origins	313	313	0	exact
1	jsd_mean	0.04212284605	0.04212284605	0	exact
1	jsd_weighted_mean	0.03740623918	0.03740623918	0	exact
2	method	coefficient	coefficient	0	exact
2	n_origins	313	313	0	exact
2	jsd_mean	0.04214861554	0.04214861554	0	exact
2	jsd_weighted_mean	0.03769727553	0.03769727553	0	exact
3	method	bayes_gravity_education	bayes_gravity_education	0	exact
3	n_origins	313	313	0	exact
3	jsd_mean	0.2838345641	0.2838345641	0	exact
3	jsd_weighted_mean	0.2650893175	0.2650893175	0	exact
4	method	unadjusted	unadjusted	0	exact
4	n_origins	313	313	0	exact
4	jsd_mean	0.04214861079	0.04214861079	0	exact
4	jsd_weighted_mean	0.03769727195	0.03769727195	0	exact

Acceptance tolerance: 1×10^{-6}.

8.4.5 `level5.structure_summary.csv`

R code: `validate_flows.R`; `validate_flow_methods.R`; `distribution_metrics.R`
Python code: `validate_flows.py`; `validate_flow_methods.py`; `distribution_metrics.py`
Test: `check_article_parity.py`; `test_validation_core.py`; `test_validation_distribution.py`; `test_validation_residuals.py`

Row	Field	R output	Python output	$\|\Delta\|$	Result
1	method	inverse_penetration	inverse_penetration	0	exact
1	n_areas	313	313	0	exact
1	pearson_residual_ benchmark_flow	0.4360944171	0.4360944171	0	exact
1	moran_i	0.08682431141	0.08682431141	0	exact
1	pearson_residual_covariate	0.09539748721	0.09539748721	0	exact
2	method	coefficient	coefficient	0	exact
2	n_areas	313	313	0	exact
2	pearson_residual_ benchmark_flow	0.2774919662	0.2774919662	0	exact
2	moran_i	0.4109623	0.4109623	0	exact
2	pearson_residual_covariate	-0.04791752542	-0.04791752542	0	exact
3	method	bayes_gravity_education	bayes_gravity_education	0	exact
3	n_areas	313	313	0	exact
3	pearson_residual_ benchmark_flow	0.4982634494	0.4982634494	0	exact
3	moran_i	0.4124461222	0.4124461222	0	exact
3	pearson_residual_covariate	0.658087603	0.658087603	0	exact
4	method	unadjusted	unadjusted	0	exact
4	n_areas	313	313	0	exact
4	pearson_residual_ benchmark_flow	0.999937528	0.999937528	0	exact
4	moran_i	0.2901810642	0.2901810642	0	exact
4	pearson_residual_covariate	-0.04730896388	-0.04730896388	0	exact

Acceptance tolerance: 1×10^{-6}.

Bayesian boundary for v07. The article harness loads the same versioned precomputed Bayesian files in R and Python. Downstream v07 validation is rerun on those rows. Independent model fitting is demonstrated separately by the controlled 12-case Bayesian audit, not by these vignette files.

9 Advanced Bayesian Adjustment

R code: `example-data.R`; `adjust_multilevel_bayes.R`
Python code: `example_data.py`; `adjust_multilevel_bayes.py`
Test: `check_article_parity.py`; `test_multilevel_frequentist_safety.py`

Source vignette: https://de-bias.github.io/debiasR/articles/v09-advanced-bayesian-adjustment.html

Checks the full-LAD advanced inputs and calls using model_engine='frequentist', the maximum-likelihood API selector, plus a static Python backend inventory. The article harness does not execute MCMC.

9.1 Complete output inventory

R code: `example-data.R`; `adjust_multilevel_bayes.R`
Python code: `example_data.py`; `adjust_multilevel_bayes.py`
Test: `check_article_parity.py`; `test_multilevel_frequentist_safety.py`

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
`data_counts.csv`	5	3	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`mpd_od.head.csv`	6	8	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`coverage.head.csv`	6	6	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`covariates.head.csv`	6	10	1×10^{-10}	1×10^{-8}	exact values	**PASS**
`distance.head.csv`	6	4	1×10^{-10}	1×10^{-8}	exact values	**PASS**

Output	Rows	Cols	Max delta	Tolerance	Equality class	Result
benchmark.head.csv	6	6	1×10^{-10}	1×10^{-8}	exact values	**PASS**
coverage_offset_frequentist.head.csv	6	7	1×10^{-5}	1×10^{-5}	within tolerance	**PASS**
reduced _ form _ frequentist . head . csv	6	5	2	2	within tolerance	**PASS**
bayesian_precomputed.output.csv	5	15	1×10^{-10}	1×10^{-8}	exact values	**PASS**
bayesian_precomputed.summary.csv	3	3	–	1×10^{-8}	exact values	**PASS**
bayesian_precomputed.diagnostics.csv	40	4	–	1×10^{-8}	exact values	**PASS**
bayesian_backend_contract.csv	2	5	–	1×10^{-8}	exact values	**PASS**

Figures. This vignette section is table/input oriented; the declared article audit contains no data-figure contract for it.

9.2 Selected side-by-side table values

R code: example-data.R; adjust_multilevel_bayes.R
Python code: example_data.py; adjust_multilevel_bayes.py
Test: check_article_parity.py; test_multilevel_frequentist_safety.py

These are the main human-readable outputs. The complete artifact inventory above accounts for every paired CSV; full files remain under audit/goldens/r/articles and audit/goldens/python/articles.

9.2.1 data_counts.csv

R code: example-data.R; adjust_multilevel_bayes.R
Python code: example_data.py; adjust_multilevel_bayes.py
Test: check_article_parity.py; test_multilevel_frequentist_safety.py

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	object	mpd_od	mpd_od	0	exact
1	rows	97969	97969	0	exact
1	columns	8	8	0	exact
2	object	coverage	coverage	0	exact
2	rows	313	313	0	exact
2	columns	6	6	0	exact
3	object	covariates	covariates	0	exact
3	rows	313	313	0	exact
3	columns	10	10	0	exact
4	object	distance	distance	0	exact
4	rows	97969	97969	0	exact
4	columns	4	4	0	exact
5	object	benchmark_od	benchmark_od	0	exact
5	rows	97969	97969	0	exact
5	columns	6	6	0	exact

Acceptance tolerance: 1×10^{-8}.

9.2.2 coverage_offset_frequentist.head.csv

R code: example-data.R; adjust_multilevel_bayes.R
Python code: example_data.py; adjust_multilevel_bayes.py
Test: check_article_parity.py; test_multilevel_frequentist_safety.py

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	origin	E06000001	E06000001	0	exact
1	destination	E06000001	E06000001	0	exact
1	flow	854	854	0	exact
1	flow_adj	14139.50462	14139.50462	0	exact
1	flow_mpd_pred	681.5103457	681.5103458	1×10^{-7}	within
1	flow_true_pred	14139.50462	14139.50462	0	exact
1	observation_probability	0.04819902564	0.04819902564	0	exact
2	origin	E06000001	E06000001	0	exact
2	destination	E06000002	E06000002	0	exact
2	flow	33	33	0	exact
2	flow_adj	116.6679712	116.6679711	1×10^{-7}	within
2	flow_mpd_pred	5.623282537	5.623282528	9×10^{-9}	within
2	flow_true_pred	116.6679712	116.6679711	1×10^{-7}	within
2	observation_probability	0.04819902564	0.04819902564	0	exact
3	origin	E06000001	E06000001	0	exact
3	destination	E06000003	E06000003	0	exact
3	flow	14	14	0	exact
3	flow_adj	101.0051374	101.0051374	0	exact
3	flow_mpd_pred	4.868349207	4.868349208	1×10^{-9}	within
3	flow_true_pred	101.0051374	101.0051374	0	exact
3	observation_probability	0.04819902564	0.04819902564	0	exact
4	origin	E06000001	E06000001	0	exact
4	destination	E06000004	E06000004	0	exact
4	flow	95	95	0	exact
4	flow_adj	138.4950271	138.495027	1×10^{-7}	within
4	flow_mpd_pred	6.675325364	6.675325358	6×10^{-9}	within
4	flow_true_pred	138.4950271	138.495027	1×10^{-7}	within
4	observation_probability	0.04819902564	0.04819902564	0	exact
5	origin	E06000001	E06000001	0	exact
5	destination	E06000005	E06000005	0	exact
5	flow	15	15	0	exact
5	flow_adj	99.81156424	99.81156412	1.2×10^{-7}	within
5	flow_mpd_pred	4.810820144	4.810820138	6×10^{-9}	within
5	flow_true_pred	99.81156424	99.81156412	1.2×10^{-7}	within
5	observation_probability	0.04819902564	0.04819902564	0	exact
6	origin	E06000001	E06000001	0	exact
6	destination	E06000006	E06000006	0	exact
6	flow	0	0	0	exact
6	flow_adj	83.86638107	83.86638094	1.3×10^{-7}	within
6	flow_mpd_pred	4.042277851	4.042277845	6×10^{-9}	within
6	flow_true_pred	83.86638107	83.86638094	1.3×10^{-7}	within
6	observation_probability	0.04819902564	0.04819902564	0	exact

Acceptance tolerance: 1×10^{-5}.

9.2.3 `reduced_form_frequentist.head.csv`

R code: `example-data.R`; `adjust_multilevel_bayes.R`
Python code: `example_data.py`; `adjust_multilevel_bayes.py`
Test: `check_article_parity.py`; `test_multilevel_frequentist_safety.py`

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	origin	E06000001	E06000001	0	exact

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	destination	E06000001	E06000001	0	exact
1	flow	854	854	0	exact
1	flow_adj	1395706302	1395706304	2	within
1	flow_mpd_pred	NA	NA	0	exact
2	origin	E06000001	E06000001	0	exact
2	destination	E06000002	E06000002	0	exact
2	flow	33	33	0	exact
2	flow_adj	11596630.23	11596630.22	1×10^{-2}	within
2	flow_mpd_pred	NA	NA	0	exact
3	origin	E06000001	E06000001	0	exact
3	destination	E06000003	E06000003	0	exact
3	flow	14	14	0	exact
3	flow_adj	10043933.85	10043933.87	2×10^{-2}	within
3	flow_mpd_pred	NA	NA	0	exact
4	origin	E06000001	E06000001	0	exact
4	destination	E06000004	E06000004	0	exact
4	flow	95	95	0	exact
4	flow_adj	13770281.03	13770281.03	0	exact
4	flow_mpd_pred	NA	NA	0	exact
5	origin	E06000001	E06000001	0	exact
5	destination	E06000005	E06000005	0	exact
5	flow	15	15	0	exact
5	flow_adj	9936052.612	9936052.612	0	exact
5	flow_mpd_pred	NA	NA	0	exact
6	origin	E06000001	E06000001	0	exact
6	destination	E06000006	E06000006	0	exact
6	flow	0	0	0	exact
6	flow_adj	8333877.312	8333877.309	3×10^{-3}	within
6	flow_mpd_pred	NA	NA	0	exact

Acceptance tolerance: 2.

9.2.4 `bayesian_precomputed.summary.csv`

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	$\lvert\Delta\rvert$	Result
1	observation_model	coverage_offset	coverage_offset	0	exact
1	real_data_evidence	The LAD example uses observed MPD flows, active-user coverage, area c...	The LAD example uses observed MPD flows, active-user coverage, area c...	0	exact
1	output_interpretation	‘flow_adj‘ is the true-flow-scale prediction and equals ‘flow_true_pr...	‘flow_adj‘ is the true-flow-scale prediction and equals ‘flow_true_pr...	0	exact
2	observation_model	reduced_form	reduced_form	0	exact
2	real_data_evidence	Real LAD S1 applications use the same OD identifiers, covariates and ...	Real LAD S1 applications use the same OD identifiers, covariates and ...	0	exact
2	output_interpretation	‘flow_adj‘ is an MPD-scale counterfactual with the fitted bias term n...	‘flow_adj‘ is an MPD-scale counterfactual with the fitted bias term n...	0	exact
3	observation_model	latent_two_level	latent_two_level	0	exact

Row	Field	R output	Python output	|Δ|	Result
3	real_data_evidence	Repeated-source S3/S4 applications use source/time MPD rows, active-u...	Repeated-source S3/S4 applications use source/time MPD rows, active-u...	0	exact
3	output_interpretation	‘latent_flow_id‘ identifies the shared OD or OD-time state. ‘flow_adj...	‘latent_flow_id‘ identifies the shared OD or OD-time state. ‘flow_adj...	0	exact

Acceptance tolerance: 1×10^{-8}.

9.2.5 `bayesian_backend_contract.csv`

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Row	Field	R output	Python output	|Δ|	Result
1	scope	nonlatent	nonlatent	0	exact
1	status	implemented	implemented	0	exact
1	implementation	Bambi/PyMC	Bambi/PyMC	0	exact
1	model_families	poisson|negbin|zip|zinb	poisson|negbin|zip|zinb	0	exact
1	evidence_scope	Static Python backend inventory; this article harness does not execut...	Static Python backend inventory; this article harness does not execut...	0	exact
2	scope	latent_two_level	latent_two_level	0	exact
2	status	implemented	implemented	0	exact
2	implementation	same debiasR Stan program via CmdStanPy	same debiasR Stan program via CmdStanPy	0	exact
2	model_families	poisson|negbin	poisson|negbin	0	exact
2	evidence_scope	Static Python backend inventory and pinned Stan-source identity; this...	Static Python backend inventory and pinned Stan-source identity; this...	0	exact

Acceptance tolerance: 1×10^{-8}.

Bayesian boundary for v09. The article harness loads the same versioned precomputed Bayesian files in R and Python. Downstream v07 validation is rerun on those rows. Independent model fitting is demonstrated separately by the controlled 12-case Bayesian audit, not by these vignette files.

10 Portable OD and benchmark workflow

R code: `measure_bias.R`; `adjust_inverse_penetration.R`; `validate_flows.R`
Python code: `benchmark_od.py`; `example_data.py`; `measure_bias.py`; `adjust_inverse_penetration.py`; `validate_flows.py`
Test: `test_example_data_companion.py`; `test_r_adjustment_parity.py`

Census is the worked example, not a required input format. The same public measurement, adjustment and validation functions accept compatible pandas tables with arbitrary area identifiers. The worked runner demonstrates this using both the full Census example and unrelated zone keys in regression tests.

Input	Required meaning
Observed OD	origin, destination, observed flow count
Benchmark OD	matching geographic/time/counting units and benchmark flow
Coverage	origin, benchmark population or margin, observed user/count denominator
Optional keys	source and time identifiers for repeated observations

```
python examples/benchmark_od.py --output output/census-od
python examples/benchmark_od.py --observed observed.csv \
  --benchmark benchmark.csv --coverage coverage.csv \
  --output output/my-od
```

Column-name options support other CSV schemas. Area/source/time identifiers are read as strings to preserve leading zeros. The caller-supplied route has no LAD/MSOA code requirement. The Census-specific companion loader validates its known geography; the underlying methods remain geography-independent.

The full-LAD run used 42455 observed OD rows and retained all adjusted rows. Validation used 34762 matched benchmark pairs. Match coverage must be reported with metrics. This example preserves zero flows and retains undefined adjusted values; if they occur, it withholds metrics rather than silently reducing the sample. Unobserved cells are not automatically interpreted as zero.

The Census example's weights use OD-margin coverage derived as in R. Since the benchmark also informs these weights, the resulting metric improvement is a worked calculation, not independent out-of-sample evidence. Another study must justify its coverage denominator, benchmark independence, units and time scope.

The integration regression suite passed 339 tests with one optional sampling test skipped. Ruff and formatting passed. These tests cover the stated cases; wider runtime versions, arbitrary external schemas and all model options are not certified by this worked example.

11 Claim boundary and exceptions

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

- 149 of 151 checks pass. The two failures concern the export table and its row-count schema, reflecting the documented organisation API policy.
- The export-list table differs intentionally; all other CSV comparisons pass under their declared raw-precision tolerances.
- The two v09 full-LAD frequentist outputs are numerically equivalent within their predeclared tolerances, not numerically identical in every cell.
- The v04 and v05 pairs use the full 331-area inputs. The three v07 pairs use the complete committed summary tables.
- ggplot2 and Matplotlib produce different graphics objects and pixels. This report compares data, variables, formulas, axes and interpretation.
- v07 pair-level scatter/band figures are listed by their exact contracts but are not newly rendered because the full pair-level plotting rows are not committed.
- The optional v07 LISA map is outside this report because the article audit excludes Local Moran permutation rows and no pinned boundary artifact is supplied.
- Bayesian vignette files are precomputed fixtures. The separate Bayesian audit previously ran fresh controlled R/Python fits. Its latest retained reference run has six convergence failures; it does not certify this integration branch.

A Complete 151-check inventory

R code: `check_article_parity.R`
Python code: `check_article_parity.py`
Test: `check_article_parity.py`

Artifact	Kind	R rows	Python rows	Max delta	Tolerance	Result
index.public_exports.csv	csv	33	32	–	1×10^{-8}	**FAIL**
index.workflow_pages.csv	csv	7	7	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.data_counts.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.input_schemas.csv	csv	27	27	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.lad_OD_travel2work.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.census_lad_OD_travel2work.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.coverage_lad.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.lad_covariates.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v03-getting-set-up.lad_centroids.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.coverage_lad.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.coverage_renamed.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.measure_bias.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.coverage_totals.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.lowest_bias.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.distribution.summary.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.distribution.top_share_diff.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.residual.summary.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.residual.definitions.csv	csv	4	4	–	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.residual.area_head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v04-measuring-coverage-bias.plot_contracts.csv	csv	4	4	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.coverage.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.covariates.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.bias.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.residual.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.bias_covariates.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.coefficient_table.csv	csv	7	7	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.fitted_residual.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.top_unexplained.csv	csv	10	10	1×10^{-10}	1×10^{-8}	**PASS**
v05-identifying-and-explaining-bias.plot_contracts.csv	csv	3	3	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.raw_heads.OD_travel2work.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.raw_heads.census_OD_travel2work.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.raw_heads.coverage.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.example_counts.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.inverse_penetration.preview.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.selection_rate.preview.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.selection_rate.attrs.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.selection_rate2.preview.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.selection_rate2.attrs.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.raking_ratio.preview.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.raking_ratio.summary.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.coefficient.preview.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.coefficient.attrs.csv	csv	1	1	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.bayesian_precomputed.display.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v06-adjusting-biases.bayesian_precomputed.metadata.csv	csv	12	12	–	1×10^{-8}	**PASS**
v06-adjusting-biases.bayesian_precomputed.model_terms.csv	csv	16	16	–	1×10^{-8}	**PASS**
v06-adjusting-biases.adjust_all_methods.summary.csv	csv	6	6	1×10^{-6}	1×10^{-6}	**PASS**
v07-validation.example_counts.csv	csv	5	5	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.method_set.csv	csv	7	7	–	1×10^{-6}	**PASS**
v07-validation.bayesian_precomputed.display.csv	csv	35	35	1×10^{-6}	1×10^{-6}	**PASS**
v07-validation.bayesian_precomputed.metadata.csv	csv	7	7	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.level1.overall_metrics.csv	csv	7	7	1×10^{-5}	1×10^{-5}	**PASS**
v07-validation.level2.origin_margin_metrics.csv	csv	4	4	2×10^{-4}	2×10^{-4}	**PASS**
v07-validation.level2.destination_margin_metrics.csv	csv	4	4	2×10^{-4}	2×10^{-4}	**PASS**
v07-validation.level3.residual_summary.csv	csv	4	4	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.level3.top_worst.csv	csv	20	20	1×10^{-6}	1×10^{-6}	**PASS**

Artifact	Kind	R rows	Python rows	Max delta	Tolerance	Result
v07-validation.level4.distribution_summary.csv	csv	4	4	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.level4.origin_head.csv	csv	20	20	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.level5.neighbor_summary.csv	csv	1	1	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.level5.structure_summary.csv	csv	4	4	1×10^{-10}	1×10^{-6}	**PASS**
v07-validation.plot_contracts.csv	csv	7	7	–	1×10^{-6}	**PASS**
v09-advanced-bayesian-adjustment.data_counts.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.mpd_od.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.coverage.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.covariates.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.distance.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.benchmark.head.csv	csv	6	6	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.coverage_offset_frequentist.head.csv	csv	6	6	1×10^{-5}	1×10^{-5}	**PASS**
v09-advanced-bayesian-adjustment.reduced_form_frequentist.head.csv	csv	6	6	2	2	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.output.csv	csv	5	5	1×10^{-10}	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.summary.csv	csv	3	3	–	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.diagnostics.csv	csv	40	40	–	1×10^{-8}	**PASS**
v09-advanced-bayesian-adjustment.bayesian_backend_contract.csv	csv	2	2	–	1×10^{-8}	**PASS**
index.meta.json	json	-	-	–	–	**PASS**
v03-getting-set-up.meta.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.meta.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.meta.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.meta.json	json	-	-	–	–	**PASS**
v07-validation.meta.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.meta.json	json	-	-	–	–	**PASS**
index.public_exports.schema.json	json	-	-	–	–	**FAIL**
index.workflow_pages.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.data_counts.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.input_schemas.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.lad_OD_travel2work.head.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.census_lad_OD_travel2work.head.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.coverage_lad.head.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.lad_covariates.head.schema.json	json	-	-	–	–	**PASS**
v03-getting-set-up.lad_centroids.head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.coverage_lad.head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.coverage_renamed.head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.measure_bias.head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.coverage_totals.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.lowest_bias.head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.distribution.summary.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.distribution.top_share_diff.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.residual.summary.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.residual.definitions.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.residual.area_head.schema.json	json	-	-	–	–	**PASS**
v04-measuring-coverage-bias.plot_contracts.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.coverage.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.covariates.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.bias.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.residual.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.bias_covariates.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.coefficient_table.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.fitted_residual.head.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.top_unexplained.schema.json	json	-	-	–	–	**PASS**
v05-identifying-and-explaining-bias.plot_contracts.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.raw_heads.OD_travel2work.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.raw_heads.census_OD_travel2work.schema.json	json	-	-	–	–	**PASS**

Artifact	Kind	R rows	Python rows	Max delta	Tolerance	Result
v06-adjusting-biases.raw_heads.coverage.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.example_counts.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.inverse_penetration.preview.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.selection_rate.preview.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.selection_rate.attrs.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.selection_rate2.preview.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.selection_rate2.attrs.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.raking_ratio.preview.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.raking_ratio.summary.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.coefficient.preview.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.coefficient.attrs.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.bayesian_precomputed.display.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.bayesian_precomputed.metadata.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.bayesian_precomputed.model_terms.schema.json	json	-	-	–	–	**PASS**
v06-adjusting-biases.adjust_all_methods.summary.schema.json	json	-	-	–	–	**PASS**
v07-validation.example_counts.schema.json	json	-	-	–	–	**PASS**
v07-validation.method_set.schema.json	json	-	-	–	–	**PASS**
v07-validation.bayesian_precomputed.display.schema.json	json	-	-	–	–	**PASS**
v07-validation.bayesian_precomputed.metadata.schema.json	json	-	-	–	–	**PASS**
v07-validation.level1.overall_metrics.schema.json	json	-	-	–	–	**PASS**
v07-validation.level2.origin_margin_metrics.schema.json	json	-	-	–	–	**PASS**
v07-validation.level2.destination_margin_metrics.schema.json	json	-	-	–	–	**PASS**
v07-validation.level3.residual_summary.schema.json	json	-	-	–	–	**PASS**
v07-validation.level3.top_worst.schema.json	json	-	-	–	–	**PASS**
v07-validation.level4.distribution_summary.schema.json	json	-	-	–	–	**PASS**
v07-validation.level4.origin_head.schema.json	json	-	-	–	–	**PASS**
v07-validation.level5.neighbor_summary.schema.json	json	-	-	–	–	**PASS**
v07-validation.level5.structure_summary.schema.json	json	-	-	–	–	**PASS**
v07-validation.plot_contracts.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.data_counts.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.mpd_od.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.coverage.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.covariates.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.distance.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.benchmark.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.coverage_offset_frequentist.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.reduced_form_frequentist.head.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.output.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.summary.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.bayesian_precomputed.diagnostics.schema.json	json	-	-	–	–	**PASS**
v09-advanced-bayesian-adjustment.bayesian_backend_contract.schema.json	json	-	-	–	–	**PASS**

B Bayesian reference quality and fresh empirical check

R code: `adjust_multilevel_bayes.R`; `run_empirical_bayes.R`; `run_empirical_offset.R`
Python code: `adjust_multilevel_bayes.py`; `_bayesian.py`; `run_empirical_bayes.py`; `run_empirical_offset.py`
Test: `recheck_bayesian_reference.py`; `test_bayesian_backend_contract.py`

The retained 12-case, three-seed R reference diagnostics were rechecked with unchanged quality thresholds: 462 checks pass and six fail. This recheck did not sample new models. Every failure belongs to the random-origin ZIP case `s1_reduced_form_zip_brms_random_origin`, seed 20260712.

Diagnostic	Observed	Required
Divergence rate	76/12000 = 0.6333%	at most 0.5%
Parameter MCSE/SD	0.07790	at most 0.05
Parameter tail ESS	137.78	at least 400
Aggregate MCSE/SD	0.08199	at most 0.05
Aggregate tail ESS	125.26	at least 400
Aggregate R-hat	1.010054	at most 1.01

The earlier full controlled run reported 2576 passes and six R-reference failures; Python quality and cross-language comparison gates passed then. That is historical evidence for the earlier translation, not a new full sampling PASS for this reorganised package. R's default brms sampler tuning is not exposed by the pinned public wrapper. Model mathematics and thresholds were preserved; accepting a new reference configuration needs explicit review.

Machine-readable reference failures and empirical summaries are recorded in `docs/reports/bayesian_checks.json`.

Two new empirical probes use the same five-area Census/MPD subset (25 OD pairs) in R and Python: Poisson, no random effects, four chains, 2000 iterations (1000 warmup and 1000 retained draws per chain), seed 20260710, and `log_pop_o + log_pop_d + log_distance`. R is rstanarm; Python is Bambi/PyMC with the mapped prior contract. CSV observation flags were restored to logical values before the R call. No prior or target was changed to obtain agreement.

Quantity	R	Python
Coverage-offset total mean	657123.20	657146.66
Coverage-offset parameter max R-hat	1.0077	1.0076
Coverage-offset minimum tail ESS	1125.22	1309.72
Coverage-offset divergences	0	0
Reduced-form total mean	2.7112×10^{21}	9.0263×10^{19}
Reduced-form parameter max R-hat	1.0027	1.0032
Reduced-form minimum tail ESS	1433.02	1329.92
Reduced-form divergences	0	0

The coverage-offset comparison sets `bias_formula= 0` and targets true flow with fixed origin coverage. Its total means differ by 0.00357%; maximum pair-mean difference is 0.814%. Python total-flow MCSE is 68.14, approximately 0.0104% of its estimated mean. These are descriptive results for one subset/seed, not a replacement for the complete declared Bayesian gate.

The reduced-form comparison removes `bias_e_origin` to target the zero-bias counterfactual. Its adjusted-flow posterior is extremely heavy-tailed; maximum pair-mean difference is 96.75%. Python total-flow MCSE is 3.0176×10^{19}, approximately 33.4% of its estimated mean. This probe is unresolved and must not be reported as numerical parity. Similar coefficient means and acceptable parameter convergence do not certify precision of an extrapolated adjusted-flow mean. The integration result now reports adjusted-total R-hat, ESS and MCSE, including MCSE relative to its mean, using a streamed draw sum so the full draw-by-OD matrix need not be retained. Posterior calculations, priors and sampler settings are preserved. Choosing a different summary or model is a scientific decision for review.

The numerical vignette comparisons above include freshly executed full-LAD frequentist models. The article's Bayesian CSVs remain precomputed fixtures; checking them cannot establish convergence of a new sampler run.

C Reduced-form investigation: priors and prediction

R code: `adjust_multilevel_bayes.R`; `check_bayesian_parity.R`
Python code: `_bayesian.py`; `adjust_multilevel_bayes.py`; `check_bayesian_parity.py`
Test: `test_bayesian_prediction_r_parity.py`; `test_bayesian_backend_contract.py`

Finding: the checked Poisson case reproduces R's priors and prediction code, while its counterfactual mean remains poorly determined. This conclusion covers the five-area, fixed-effect reduced-form model; it does not certify other Bayesian families or resolve the separate random-origin ZIP reference failures.

Both implementations use the same 25 fitting rows, outcome, transformed predictors and formula. The resolved Normal prior locations and scales match: centred intercept scale 2.5; slopes 2.5/sample_sd. The bias slope scale is approximately 985.16 because observed bias varies very little.

The reproducible checker evaluates the centred log prior and Poisson log likelihood at identical R coefficient draws, then sends those draws through the actual R and Python prediction helpers. It checks both observed predictions and the zero-bias counterfactual. Joint log priors agree within 1.5×10^{-13}, log likelihoods within 5.8×10^{-11} and predicted rates within 1.6×10^{-13} relative error. This rules out the checked prior parameters, centring, link and counterfactual prediction paths as the explanation for this discrepancy.

Cause and sensitivity. Observed bias lies between 0.9644 and 0.9714. Zero is approximately 381 sample standard deviations from its mean. Removing a fitted bias coefficient near -30 multiplies rates by roughly exp(29). Coefficient uncertainty becomes extreme uncertainty on the flow scale. The observed total is 21488; the median of R's counterfactual total draws is already approximately 1.15×10^{17}. In the first R run, one draw supplies 74.8% of the sum used to estimate the total mean.

Run	Retained draws	R MCSE/mean	Python MCSE/mean
Seed 20260710; 2000 iterations	4000	96.46%	33.43%
Seed 20260711; 8000 iterations	16000	21.55%	33.42%

Both runs use four chains and half the iterations for warmup. The second changes seed and draw count together, so it demonstrates sensitivity rather than isolating draw-count effects. Shared-draw checks pass in both; independent mean differences remain large. Maximum pair-median differences are 20.1% and 8.7%. Medians have not been substituted for R's default mean.

Fix and limit. Python now records bias fit ranges, counterfactual support and the largest draw's share of the sum. A requested mean emits a RuntimeWarning when total-flow MCSE exceeds 5% of its magnitude. This explicit Python precision warning preserves the posterior and estimator; it is not an R default or a relaxed parity gate. Its absence does not certify individual flows, convergence or model validity. Median precision is not assessed by it.

The investigation and commands are documented in `docs/development/bayesian-investigation.md`; compact evidence is retained in `docs/reports/bayesian_checks.json`. The new live-R regression passes. The full suite has 288 passes and one optional sampling skip at that investigation revision; Ruff and formatting pass. The full Bayesian matrix was not rerun. No stronger prior, clipping or summary change was used to make the result pass. The counterfactual interpretation still needs maintainer review. Main remains untouched.

D Non-Bayesian issue completion and cleanup

R code: `distribution_metrics.R`; `validate_flows.R`; `validate_flow_methods.R`; `plot_validation.R`
Python code: `distribution_metrics.py`; `validate_flows.py`; `validate_flow_methods.py`; `plot_validation.py`
Test: `test_shared_distribution_metrics.py`; `test_flow_residual_structure.py`; `test_validate_flow_methods.py`; `test_lisa_plot.py`

Checked update: 13 October 2026. The four identified non-Bayesian gaps are implemented and tested on the integration branch. GitHub issues 12–15 remain open pending human review and focused PRs. Each change is a separate commit, followed by a cleanup commit. Main remains untouched.

The cleaned package passes **339 tests**, with one optional sampling test skipped. Ruff, formatting and whitespace checks pass. The article audit was rerun against retained outputs from the unchanged R reference: **149 of 151 checks pass**; both remaining differences concern intentional public exports. All compared scientific tables retain their existing tolerances.

D.1 Issue 12: shared distribution calculations

R code: `distribution_metrics.R`; `measure_bias.R`; `validate_flows.R`
Python code: `distribution_metrics.py`; `measure_bias.py`; `validate_flows.py`
Test: `test_shared_distribution_metrics.py`

Smoothing, shares, KL contributions and Jensen–Shannon contributions are now implemented once in the shared private calculation module, reused by coverage measurement and OD validation. Tests compare their public results and verify that separate source/origin groups retain their own denominators. Zero and missing behaviour retains the established API contracts; no numeric acceptance threshold was changed.

D.2 Issue 13: spatial residual diagnostics

R code: `validate_flows.R`
Python code: `validate_flows.py`
Test: `test_flow_residual_structure.py`

The new regression exposed a real classification error. Significance flags were converted to numeric 0/1, while the classifier accepted only booleans. Significant areas were therefore labelled "not significant". Flags now use nullable booleans. Fixed global and Local Moran calculations match R for clustered, weighted, constant-residual and isolated-area cases. Endpoint roles, aggregation, covariates, geometry and invalid spatial inputs are covered.

Area	R Local Moran	Python Local Moran	R cluster	Python cluster
A	1.25	1.25	low-low	low-low
B	1.25	1.25	low-low	low-low
C	1.25	1.25	high-high	high-high
D	1.25	1.25	high-high	high-high
E	NA	NA	no neighbours	no neighbours

This is the synthetic five-area regression fixture from R's tests, with 19 permutations and alpha 1 to check classification. It is not evidence of empirical statistical significance. Python seeded permutations are reproducible within Python; identical R/NumPy permutation sequences are not claimed.

D.3 Issue 14: multi-method validation

R code: `validate_flow_methods.R`
Python code: `validate_flow_methods.py`
Test: `test_validate_flow_methods.py`

Preparation and validation wrappers reuse the core calculations and preserve method names. Errors now identify the failing method while retaining the underlying exception type. Tests cover multiple methods, margin totals, selected subsets, custom columns, conflicting inputs, non-data-frame values, reserved columns and ambiguous repeated-observation joins.

D.4 Issue 15: validation and LISA plots

R code: `plot_validation.R`
Python code: `plot_validation.py`
Test: `test_lisa_plot.py`; `test_validation_plot_contract.py`

All eight plotting functions have executable coverage. Actual GeoPandas LISA figures render and save successfully. Tests verify cluster colours, significance filtering, isolates, unmatched polygons, method/comparison panels, labels, ordering and outlines. Cluster colours now follow R's defaults consistently across panels. Outlines are reprojected to the boundary CRS when both systems are known and differ. Duplicate classifications within one map panel raise clearly. Pixel-identical rendering is not claimed.

D.5 Legacy and metadata cleanup

R code: `example-data.R`; `adjust_multilevel_bayes.R`
Python code: `example_data.py`; `_latent.py`; `_bayesian_latent.py`
Test: `test_example_data_companion.py`; `test_latent_backend_contract.py`; `test_multilevel_frequentist_safety.py`

The cleanup removes 471 net lines: the R data-loader/downloader, production synthetic fixture mode, unused source-search branches, derived fallback covariates/distances, runtime SHA fingerprints and hashed cache layout. Synthetic fixtures live in tests. The Python companion is the empirical route; explicit OD paths still require its covariate/centroid assets. Supplied-file metadata identifies the filename. Incomplete centroid coverage raises instead of silently returning empty distances.

The Stan model is unchanged and still compared byte-for-byte with R in tests. Cached source uses a direct byte comparison to detect changes. Scientific output schemas, source attribution, model settings, diagnostics, contributor records and the single MIT licence are retained. Obsolete approval labels and duplicate import scaffolding are removed. Polars remains a planned evaluation, not a new dependency or changed public API.

A rebuilt wheel runs the synthetic and full Census LAD examples outside the checkout. The Census run retains 42455 observed/adjusted rows and validates 34762 matched benchmark pairs. These checks use prepared dependencies; broader runtime/dependency matrices, Quarto rendering and fresh CmdStan compilation/ sampling were not exercised. The reduced-form Bayesian uncertainty and retained R ZIP reference failures remain unresolved. The implementation and cleanup require human review under the existing contribution workflow; no issue was closed and no main merge was performed.

E Simplification and clean installation

R code: adjust_multilevel_bayes.R; example-data.R
Python code: adjust_multilevel_bayes.py; _multilevel_inputs.py; _frequentist.py; example_data.py
Test: test_bayesian_backend_contract.py; test_multilevel_frequentist_safety.py; test_example_data_companion.py

Adjustment, validation and plotting methods use the mapped public Python modules. The contribution checks identify the tested package and its prerequisites. Runtime Python source decreases by 472 lines compared with commit 6568831. Moving fitting and shared input preparation into their own private modules reduces the public multilevel module from 2094 lines to 524; the mapped public function remains in place.

E.1 Removed legacy paths and repeated code

R code: adjust_multilevel_bayes.R; example-data.R
Python code: _bayesian.py; _multilevel_inputs.py; example_data.py; plot_validation.py
Test: test_bayesian_backend_contract.py; test_example_data_companion.py; test_validation_plot_contract.py

The duplicate Bayesian formula parser, irrelevant latent-backend arguments, silent row-ID deduplication, unused plotting implementation, repeated plotting table extraction, companion filename/dataset aliases and duplicated runtime dependency declarations are removed. A further 91 runtime lines and 103 article-checker lines are removed through shared global Moran calculations, model-result attributes and recursive plotting table extraction. All original tests remain, with one added regression for nested method metrics. Input normalization rules, scientific tolerances and model diagnostics are preserved. Shared joins reject conflicting input records before fitting. Companion assets are loaded by name through its public API, including MSOA assets stored separately from LAD data.

The deprecated R `custom_formula` argument is omitted from this new Python interface; `formula` and paired mobility/bias formulas retain the model specifications. Optional installations use `grouped`, `bayesian` and `latent` extras, replacing the bundled `models` extra. Model calculations and method availability are preserved.

E.2 Installation, examples and documentation

R code: check_article_parity.R
Python code: check_article_parity.py; coverage_adjustment.py; benchmark_od.py
Test: test_bayesian_prediction_r_parity.py; test_bayesian_backend_contract.py; test_lisa_plot.py

On macOS ARM64, the current Python 3.11 wheel passes 293 non-Bayesian tests with live R. The earlier complete check passed 342 tests with short Bambi/PyMC sampling. The earlier fresh Python 3.14 base/geographic check passed 114 tests with one optional R comparison skipped. Both environments pass `pipcheck`. The installed Census example retains 42455 observed/adjusted rows, zero undefined adjustments and 34762 matched benchmark pairs.

The fresh full-stack install exposed an ArviZ plotting import failure with Matplotlib 3.11. The Bayesian extra constrains Matplotlib to versions below 3.11; the successful rerun uses 3.10.9. Base/geographic checks retain support for the newer runtime. This failure corresponds to https://github.com/arviz-devs/arviz-plots/issues/453. The non-sampling tutorial snippets execute and the existing Quarto website renders.

E.3 Reproduce the report and its checks

R code: check_article_parity.R; check_bayesian_parity.R
Python code: build_report.py; check_article_parity.py; check_bayesian_parity.py
Test: check_article_parity.py; test_bayesian_prediction_r_parity.py

The repository now contains this report's authored LaTeX and recorded figures. `build_report.py` compiles the evidence snapshot; `check_article_parity.py` runs the 151 original comparisons using fresh R and Python outputs and their original tolerances. The only expected differences concern the Python public exports. Full-precision CSV outputs and comparison JSON stay in the caller's run directory. The build command can append that fresh comparison summary to the recorded report. See `docs/reports/README.md`.

Bayesian vignette tables remain precomputed references. The separate reproducible checker investigates reduced-form priors, predictions and sampling. Full Bayesian matrix and CmdStan sampling checks remain outstanding; counterfactual uncertainty and six retained R ZIP quality failures remain unresolved. Issue-specific contribution scopes cover issues 15–7 and grouped 4–6. Human review remains pending. Suggested proposal format: `docs/development/contribution-format.md`.

F Fresh article reproduction

R code: `check_article_parity.R`

Python code: `check_article_parity.py`

Test: `check_article_parity.py`

Fresh R/Python calls passed 149 of 151 comparisons. The two differences concern public exports. Bayesian article tables use retained R vignette outputs; sampling requires the separate Bayesian investigation.

R reference: `6fa2709ae42de6dc525a4a46182fdfb448b14a3f`

Python checkout: `73932f903bc601781d5ea892015e1cfc05379b19`

The main report preserves the recorded tables and figures. The fresh run stores complete comparison CSVs alongside its JSON summary. Uncommitted changes, when present, are listed in that summary.

G Method-by-method evidence

R code: `check_article_parity.R`

Python code: `check_article_parity.py`; `build_report.py`

Test: `test_r_adjustment_parity.py`; `test_validation_core.py`

Live R executes the reference during the article checker or named regression. Retained R uses previously recorded reference outputs. Python contracts check behaviour, schemas, errors and figures. Each row identifies its checked scope. The table source is `docs/development/method-evidence.md`.

Method and sources	Evidence and tests
`measure_bias` (Measurement) R: `measure_bias.R` Python: `measure_bias.py`	Live R: Census arithmetic and zero, missing and infinite values. Python: input and warning contracts. Test: `test_measure_bias.py`; `test_measure_bias_r_parity.py`
`measure_bias_distribution` (Measurement) R: `measure_bias.R` Python: `measure_bias.py`	Live R article: Census shares and divergence summaries. Python: smoothing, invalid totals and missing values. Test: `test_measure_bias_distribution.py`; `test_shared_distribution_metrics.py`
`validate_bias_residual_structure` (Measurement) R: `measure_bias.R` Python: `measure_bias.py`	Live R article: Census residual summaries. Python: endpoint roles, weighted links and isolated areas. Test: `test_validate_bias_residual_structure.py`
`adjust_inverse_penetration` (#4) R: `adjust_inverse_penetration.R` Python: `adjust_inverse_penetration.py`	Live R: origin/destination/both weights, zero/missing/infinite values, row expansion and overwriting adjusted flow. Article: Census outputs. Test: `test_adjust_inverse_penetration.py`; `test_r_adjustment_parity.py`
`adjust_selection_rate` (#5) R: `adjust_selection_rate.R` Python: `adjust_selection_rate.py`	Live R: origin/OD calibration, missing coverage, repeated observations and numerical boundaries. Article: Census outputs. Test: `test_adjust_selection.py`; `test_r_adjustment_parity.py`
`adjust_selection_rate2` (#6) R: `adjust_selection_rate2.R` Python: `adjust_selection_rate2.py`	Live R: origin/OD calibration, missing coverage, repeated observations and numerical boundaries. Article: Census outputs. Selection-rate II also checks grouped calibration. Test: `test_adjust_selection.py`; `test_r_adjustment_parity.py`
`adjust_raking_ratio` (#7) R: `adjust_raking_ratio.R` Python: `adjust_raking_ratio.py`	Live R article: Census adjusted totals and selected rows. Python: margin conservation, source/time scope and zero-cell behaviour. Test: `test_adjust_raking.py`
`adjust_coefficient` (#8) R: `adjust_coefficient.R` Python: `adjust_coefficient.py`	Live R article: Census coefficients, totals and selected rows. Python: calibration, required columns and numerical boundaries. Test: `test_adjust_coefficient.py`
`debiaspy_example_data` (#9) R: `example-data.R` Python: `example_data.py`	Live R article: named Census asset counts/schemas and selected values. Python: companion API, LAD/MSOA inputs and portable CSV identifiers. Companion assets required for the empirical loader. Test: `test_example_data_companion.py`; `test_benchmark_od.py`
`adjust_all_methods` (#10) R: `adjust_all_methods.R` Python: `adjust_all_methods.py`	Live R article: selected-method Census summaries. Python: selection, parameters and scoped preparation. Available engine scope follows each method. Test: `test_adjust_all_methods.py`; `test_multilevel_frequentist_safety.py`
`adjust_multilevel_bayes` (#16 (deferred)) R: `adjust_multilevel_bayes.R` Python: `adjust_multilevel_bayes.py`	Frequentist: live R Census article and retained R model probes. Bayesian: retained investigation and backend contracts; acceptance remains pending. Checked Bayesian scope: fixed-effect Poisson investigation. Test: `test_multilevel_frequentist_safety.py`; `test_bayesian_prediction_r_parity.py`; `test_bayesian_backend_contract.py`; `test_latent_backend_contract.py`
`validate_flow_overall` (#11) R: `validate_flows.R` Python: `validate_flows.py`	Live R article via method wrapper: Census metrics. Python: grouped sources, missing values and joined data. Test: `test_validation_core.py`
`validate_flow_pairs` (#11) R: `validate_flows.R` Python: `validate_flows.py`	Live R article via overall/wrapper outputs: Census joined pairs. Python: signed residuals and join columns. Test: `test_validation_core.py`
`validate_flow_distribution` (#12) R: `validate_flows.R` Python: `validate_flows.py`	Live R article via method wrapper: Census divergence summaries. Python: smoothing and invalid contributions. Test: `test_validation_distribution.py`

Method and sources	Evidence and tests
`validate_flow_residuals` (#13) R: `validate_flows.R` Python: `validate_flows.py`	Live R article via method wrapper: Census residual summaries. Python: ranking, signs and missing/infinite values. Test: `test_validation_residuals.py`
`validate_flow_residual_structure` (#13) R: `validate_flows.R` Python: `validate_flows.py`	Live R regression: four spatial cases, global/Local Moran, weighted links, isolates and constant residuals. Article: Census summaries. Permutations reproduce within Python. Test: `test_flow_residual_structure.py`
`validate_flow_prepare_output` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Python contracts: single output, columns and input preservation; exercised by preparation of multiple methods. Test: `test_validate_flow_methods.py`
`validate_flow_prepare_outputs` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: preparation of Census methods. Python: named mappings, reserved columns and ambiguous keys. Test: `test_validate_flow_methods.py`
`validate_flow_overall_methods` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: per-method Census metrics. Python: forwarding and failing-method context. Test: `test_validate_flow_methods.py`
`validate_flow_residual_methods` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: per-method Census residuals. Python: forwarding and failing-method context. Test: `test_validate_flow_methods.py`
`validate_flow_distribution_methods` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: per-method Census distributions. Python: forwarding and failing-method context. Test: `test_validate_flow_methods.py`
`validate_flow_residual_structure_methods` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: per-method Census spatial summaries. Python: forwarding and failing-method context. Test: `test_validate_flow_methods.py`
`validate_flow_margins` (#14) R: `validate_flow_methods.R` Python: `validate_flow_methods.py`	Live R article: origin/destination Census margins. Retained R fixture totals and Python input-preservation contracts. Test: `test_validate_flow_methods.py`
`validate_flow_plot_metrics` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: nested metrics, labels and plotted values. Test: `test_validation_plot_contract.py`
`validate_flow_plot_residuals` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: residual values and method/comparison panels. Test: `test_validation_plot_contract.py`
`validate_flow_plot_scatter` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: series, axes and labels. Test: `test_validation_plot_contract.py`
`validate_flow_plot_residual_bands` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: bands and summary contracts. Test: `test_validation_plot_contract.py`
`validate_flow_plot_distribution` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: divergence series and ordering. Test: `test_validation_plot_contract.py`
`validate_flow_plot_distribution_pairwise` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: pairwise inputs and panels. Test: `test_validation_plot_contract.py`
`validate_flow_plot_structure` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: spatial summary panels. Test: `test_validation_plot_contract.py`
`validate_flow_plot_lisa_map` (#15) R: `plot_validation.R` Python: `plot_validation.py`	Python figure contracts: cluster colours, significance, isolates, geometry and CRS alignment. Test: `test_lisa_plot.py`; `test_validation_plot_contract.py`

H Issue contribution checks

R code: `measure_bias.R`; `adjust_inverse_penetration.R`; `validate_flows.R`

Python code: `adjust_all_methods.py`; `validate_flows.py`; `build_report.py`

Test: `test_r_adjustment_parity.py`; `test_flow_residual_structure.py`

Each branch was built and installed as a wheel, then tested with its documented prerequisites on macOS ARM64 with Python 3.11 and prepared dependencies. Live R was enabled. All exported callables, Ruff, formatting and pip dependency checks passed. The issue wheels contain the non-Bayesian implementations. Counts include prerequisite tests. The complete package retains all 204 original test functions and their assertion bodies, with 293 non-Bayesian tests passing. The complete setup uses the grouped/plot extras and pinned companion. A fresh base development/plotting setup passes 281 tests and skips nine GPBoost fitting cases and three companion-data cases. The missing-runtime error check remains active; both setups pass pip dependency checks.

Contribution and base	Installed checks and scope
`contribution/issue-4-6` Base: `main`	93 passed Inverse penetration and both selection methods; live R calibration and input boundaries.
`contribution/issue-7` Base: `contribution/issue-4-6`	104 passed Raking: source/time keys, target margins and numerical boundaries.
`contribution/issue-8` Base: `contribution/issue-7`	114 passed Coefficient calibration: OLS/count families, warning contracts and numerical references.
`contribution/issue-9` Base: `main`	51 passed Named companion LAD/MSOA assets. Portable end-to-end example tests use the complete contribution package.
`contribution/issue-11` Base: `main`	57 passed Measurement arithmetic, shared numeric helpers, paired and overall validation; scoped joins, denominators and missing values.
`contribution/issue-12` Base: `contribution/issue-11`	67 passed Distribution validation: weighted shares, KL/JSD and invalid contributions.
`contribution/issue-13` Base: `contribution/issue-12`	89 passed Residual and spatial diagnostics: live R global/Local Moran, weighted links and isolated areas.
`contribution/issue-14` Base: `contribution/issue-13`	107 passed Named multi-method wrappers, margin helpers and method-specific errors.
`contribution/issue-15` Base: `contribution/issue-14`	144 passed Eight plot functions: figure contracts, panels, LISA classes and CRS alignment.
`contribution/issue-10` Base: `contribution/issue-8`	117 passed Explicitly selected deterministic-method orchestration. Default methods="all" and multilevel dispatch require the model implementation under issue 16.